Supplementary Material for "Verification-Guided LLM Agents for Executable Optimization Modeling of Weapon–Target Assignment"

ABSTRACT

This appendix supports the main paper with the protocol definitions, full agent prompts, expanded result tables, benchmark documentation, and two complete end-to-end case traces. WTA-FormBench is released with this appendix at https://anonymous.4open.science/r/WTA-MAFA2026-C583/. Appendix K states what that release contains.

Contents

Appendix A Overview and Evidence Map 1

Appendix B Benchmarks and Experimental Protocol 2
- Appendix B.1 SWTA and MADRA composition 2
- Appendix B.2 Shared execution settings 2
- Appendix B.3 Baseline reproduction 2
- Appendix B.4 Controlled-variable and leakage audit . . 2

Appendix C Agent Prompts and Role Contracts 4
- Appendix C.1 Planner prompt 4
- Appendix C.2 Modeler prompt 4
- Appendix C.3 Critic prompt 4
- Appendix C.4 Error Router prompt 4
- Appendix C.5 Executable Builder prompt 5
- Appendix C.6 Defect taxonomy and repair ownership . . 6
- Appendix C.7 SCW construction algorithm 6

Appendix D Output Contract and Artifact Coverage 7

Appendix E Expanded SWTA and MADRA Results 8
- Appendix E.1 Stage funnels 8
- Appendix E.2 Domain-quality dimensions 8
- Appendix E.3 Modeling cost 8
- Appendix E.4 Failure endpoints 8

Appendix F Expert Reference Comparison 10

Appendix G Ablation, Backbone Sensitivity and Robustness 12
- Appendix G.1 Backbone sensitivity 12
- Appendix G.2 Preference, scale and multi-objective robustness 12

Appendix H WTA-FormBench: Construction, Boundary and Results 13
- Appendix H.1 Benchmark card 13
- Appendix H.2 Eighteen source families 13
- Appendix H.3 Source references 13
- Appendix H.4 Construction and freeze procedure 13
- Appendix H.5 Input boundary 14
- Appendix H.6 Scenario adaptation across all 18 families 14
- Appendix H.7 Cross-family separation of model vocabulary 15
- Appendix H.8 Process and boundary diagnostics 16

Appendix I Case Study I: A Repaired SWTA Success 16
- Appendix I.1 Public input: the natural-language scenario 16
- Appendix I.2 Public input: instance structure 18
- Appendix I.3 Planner output 18
- Appendix I.4 Model Design Card 19
- Appendix I.5 Critic report 20
- Appendix I.6 Error Router decision and route history . 20
- Appendix I.7 Generated executable model 21
- Appendix I.8 Model registry 22
- Appendix I.9 Solver result and decoded solution 23
- Appendix I.10 Mathematical description 24

Appendix J Case Study II: A MADRA Failure Stopped at the First Route 24
- Appendix J.1 Public input: the natural-language scenario 24
- Appendix J.2 Public input: instance structure 25
- Appendix J.3 Planner output 26
- Appendix J.4 Model Design Card 27
- Appendix J.5 Critic report 28
- Appendix J.6 Error Router decision and route history . 28
- Appendix J.7 Failure report 29
- Appendix J.8 Retained mathematical description 30

Appendix K Released Material 30

Appendix A. Overview and Evidence Map

The central claim this appendix supports is narrow and checkable: under a public-only information boundary, WTA-MAFA does not merely emit a mathematical description, but produces a model bundle that a solver can execute, a decoder can replay, an external evaluator can rescore, and, when it fails, an audit can attribute to a responsible stage. Every aggregate in the main paper is traceable to a per-instance record, and failed samples remain in the denominator throughout.

Table A.1 maps each main-text object to the appendix evidence that supports it.

Table A.1
Evidence map between the main text and this appendix.

Main-text object	Role in the main text	Appendix evidence
Table 2: end-to-end results	Modeling coverage and cross-method comparison	Appendix E, Tables E.1–E.2
Table 3: SWTA and MADRA domain quality	Domain-quality comparison and reference rescoring	Appendix E, Table E.3
Figure 5: expert reference comparison	Pairing coverage, feasibility parity and objective trade-offs	Appendix F, Tables F.1–F.2
Figure 6: ablation and backbone	Where the gain originates; backbone independence	Appendix G, Table G.1
Table 4: WTA-FormBench adaptation	Scenario breadth across 18 task families	Appendix H.6, Appendix H.7, Table H.6
Failure and robustness claims	Boundaries, failure modes, behavior under pressure	Appendix E.4, Appendix G.2, Appendix J
Public-only boundary claim	That generation never saw hidden references	Appendix B.4, Table B.5

Appendix B. Benchmarks and Experimental Protocol

Appendix B.1. SWTA and MADRA composition

Table B.1 summarizes the two end-to-end benchmarks. They are deliberately asymmetric. SWTA supplies scale and breadth, with 162 samples spanning five multi-objective profiles, two instance types and several size tags, so it measures whether the system holds up across many task variants. MADRA supplies temporal strictness: only 20 samples, but each couples radars, launchers, targets and hit times into a chain where one misplaced launch makes the whole plan infeasible. A method can score well on SWTA through breadth alone; MADRA is where temporal and resource consistency is actually tested.

SWTA [1] exposes sensors, weapons, targets, candidate engagements, target values, detection and interception effectiveness, and resource and temporal constraints. MADRA [2] exposes radars, launchers, targets, hit times, channel occupancy, inventory, and repeated-engagement intervals; it tests the radar–launcher–target–hit-time chain and multi-round interception feasibility.

Appendix B.2. Shared execution settings

Table B.2 lists the settings held fixed across every method and every sample. Three of them bound the comparison in ways worth stating explicitly. The limit of 18 LLM calls and 8 routing rounds per sample caps how much WTA-MAFA may spend recovering from its own errors, so none of its results depends on unbounded retrying; each baseline keeps its own published debugging loop, which is why OptiMUS v0.3 and ORPilot exceed 18 calls in main-text Table 2. The 300 s solver limit and 10^{-4} MIP gap are identical for all methods, so solver time never distinguishes them. And the public-input restriction means the agents see only the scenario text, the data description and the instance file, never a reference formulation.

Appendix B.3. Baseline reproduction

Table B.3 records what was preserved from each baseline and what had to be adapted. The principle throughout is that a baseline keeps its own reasoning and only its interfaces change: agent order, prompting structure, debugging policy and solver interface are left as published, while the public-input wrapper and the output packaging are replaced so that every system reads the same instance and is scored by the same evaluator. No baseline receives the Builder Team, the SCW, the Error Router, bounded Repair, or any WTA-specific feedback.

No method may use external retrieval or hidden references. Where a baseline supports retrieval augmentation it is disabled, so that every system operates inside the same public-information boundary. The OptiMUS entry is therefore a public-input-only, retrieval-free adaptation rather than a paper-faithful reproduction of the original system, and its results bound that configuration rather than the authors' intended deployment.

Screened but not run at full scale. Two further systems were adapted to the same interface and screened on a 10-sample smoke test before committing to a full run. Neither closed a single case, so we did not proceed. MM-Agent [7] returned `METHOD_TIMEOUT` on all ten samples, producing partial output but never reaching a model. ModelingAgent [8] behaved differently: all ten samples yielded a schema-valid, boundary-safe bundle, but none reached `build_success`, with nine ending in solver timeout and one in a method failure (Table B.4). The distinction matters for interpreting these screens, since ModelingAgent's failures are downstream of a well-formed artifact whereas MM-Agent never produced one, but in both cases a 0/10 screen made a 182-sample run unjustifiable, and we report them as screens rather than as comparison results.

Appendix B.4. Controlled-variable and leakage audit

Table B.5 states, for each experiment block, what was held fixed and what was allowed to vary. Reading down the column of quantities allowed to change is the quickest way to check that each claim rests on a single manipulation: the architecture ablation changes only components, the backbone study changes only the model provider, and the baseline comparison changes only the input wrapper and backend adapter.

The audit answers two questions: whether generation used only public input, and whether each comparison changed only the factor it claims to change. Boundary reports, external-only flags, and public-input digests are recorded per run; reference solutions, trusted builders, and evaluator outputs are blocked during generation. Failed samples enter the denominator, and unavailable states are recorded as unavailable rather than imputed as zero. For an automated-modeling paper this matters as much as the headline numbers: if generation could see a reference answer, no success rate afterwards would demonstrate modeling ability.

Table B.1
SWTA and MADRA benchmark composition.

Benchmark	N	Subgroups	Main modeling pressure	Role in the paper
SWTA	162	MOP1–MOP5; Type0/Type1; O2/O3/O4; multiple scales	Candidate engagement, multi-objective assignment, resource capacity, spatial and probability-dependent effects	Large-scale formal comparison
MADRA	20	P1–P10; single and multi3	Radar–launcher–target–hit-time coupling, inventory, channel capacity, hit intervals and repeated engagements	Small but temporally strict comparison

Table B.2
Shared execution settings for the SWTA and MADRA experiments.

Setting	Configuration
Operating system	Windows 10.0.26200 SP0
Python	3.9.13
Solver	Gurobi 10.0.1
CPU	Intel Core i9-14900HX
Main backbone	DeepSeek-V4-Pro
Backbone sensitivity	Qwen3.7-Max and Kimi-K3 on the fixed 40-sample slice
Public input	Public scenario description, data description and instance data only
LLM timeout	180 s per call
Global LLM hard limit (WTA-MAFA)	18 calls per sample
Maximum route rounds (WTA-MAFA)	8
Solver time limit	300 s
MIP gap	10^{-4}
Solver threads	0 (solver default)

Table B.3
Baseline reproduction matrix. All methods run on the same DeepSeek-V4-Pro backbone and are scored by the same external evaluator.

Method	Reproduced components	Benchmark-specific adaptation	Role
OR-LLM-Agent [3]	Original modeling, code-generation and debugging order	Public-input wrapper and unified external evaluation	General OR-agent comparison
CoE [4]	Chain-of-experts collaboration and trial logic	Public-input wrapper and unified external evaluation	Multi-expert modeling comparison
OptiMUS v0.3 [5]	Objective and constraint extraction, model generation, execution feedback	Public-input wrapper, unified external evaluation, retrieval augmentation disabled, adapter-only constraint normalization	Solver-feedback modeling comparison
ORPilot [6]	Official end-to-end workflow: modeling, code generation, execution and its own debugging	Public-input wrapper and unified external evaluation; a solution view is emitted only when the generated model's variables map deterministically onto public engagement options	Industrial OR-agent comparison
WTA-MAFA	Planner, Modeler, Critic, Builder Team, SCW, Error Router, bounded Repair and replay checks	Native WTA-MAFA execution contract	Proposed system

Table B.4
Ten-sample smoke screens for the two systems not run at full scale. Stage counts are out of 10.

Method	Bundle valid	Boundary safe	Build success	Full success	Failure profile
MM-Agent	–	–	0	0	10 method timeout
ModelingAgent	10	10	0	0	9 solver timeout, 1 method failure

Table B.5
Controlled-variable audit across experiment blocks.

Block	Sample scope	Fixed	Allowed to change
Formal182 C0	182 SWTA/MADRA samples	WTA-MAFA v0.4	No generation-stage reference or evaluator feedback
C-baseline	Same benchmark and evaluator	Original baseline logic retained	Public-input wrapper and model-backend adapter only
D-Arch	Representative 40-sample slice	Backbone, solver, evaluator, samples	Architecture components only
D-Backbone	Same 40-sample slice	Architecture, prompts, schema, solver, evaluator	LLM provider and model only
E reference	Paired contexts where available	Not an automated-modeling competitor	No feedback into any generation stage

Appendix C. Agent Prompts and Role Contracts

Each agent holds a frozen prompt template that fixes its role, its input, and the schema of its output. Table C.1 indexes them; the text of each follows. Together they define the role boundaries that Section 3 describes: Planner never writes a model, Modeler never writes code, Critic never repairs a design, and only the Executable Builder emits an executable artifact.

Appendix C.1. Planner prompt

```
You are the WTA-MAFA v0.4 Planner Agent. Return exactly one JSON object matching PlannerOutput:
benchmark, sample_id, public_input_digest, problem_family, public_fields, objective_candidates, constraint_candidates,
executable_risks, forbidden_data_boundaries, model_scope_assessment, provenance. Read only public input and executable data
manifests. Identify the problem family, public fields, objective candidates, constraint candidates, solver executability risks, and
forbidden data boundaries. Do not write solver code, do not request hidden references, and do not claim model authority. If
BENCHMARK_EXECUTION_CONTRACT is provided, treat its hard fields as the benchmark execution boundary. Plan inside that boundary; do
not reinterpret authoritative candidate sets, objective contracts, solution_view requirements, forbidden transformations, or
complexity limits as optional natural-language hints. Before proposing variables or constraints, decide what modeling level the
public input actually supports. Use PUBLIC_MODEL_SCOPE_DRAFT as the deterministic floor. model_scope_assessment must state
selected_modeling_level, supported_constructs, partially_supported_constructs, unsupported_constructs,
forbidden_modeling_constructs, temporal_policy, uncertainty_policy, scale_guardrails, objective_scope, and builder_directives. Do
not upgrade beyond public evidence. If launch_step or equivalent public launch timing is absent, forbid independent launch_time or
launch_step decision variables. If trajectory, position, coordinate, or velocity fields are absent, forbid trajectory/kinematic
constraints. If scenario, distribution, or uncertainty fields are absent, forbid stochastic or robust optimization. If a sparse
public candidate list exists, prefer candidate_assignment with x[e] and forbid dense Cartesian expansion unless public input proves
it is required and tractable. forbidden_data_boundaries must be a non-empty JSON array. Include at least
hidden/reference/trusted-builder boundaries and any non-public raw probability, answer-key, or benchmark skeleton fields that must
not be used. If ROUTER_FEEDBACK is present, repair only the listed PlannerOutput contract defects. Keep public_input_digest exactly
equal to PUBLIC_INPUT_DIGEST. If MEMORY_CARD is present, use it only as public-safe guidance about recurring modeling pitfalls,
scale risks, and successful formulation patterns. It is not a source of answers, not a solver-bound model, and not a substitute for
the current public input.
```

Appendix C.2. Modeler prompt

```
You are the WTA-MAFA v0.4 Modeler Agent. Return exactly one JSON object matching ModelDesignCard:
benchmark, sample_id, public_input_digest, planner_digest, variables, objectives, constraints, assumptions, solver_capability_notes,
public_data_refs, scope_commitment, provenance. Produce a mathematical design card from public input and PlannerOutput. This card is
not the authoritative solver model; it guides the Builder. Do not output source_code. Do not use hidden references, trusted
builders, benchmark skeletons, or answer keys. If BENCHMARK_EXECUTION_CONTRACT is provided, treat its hard fields as binding
execution facts. You may choose compact equivalent modeling patterns inside the contract, but do not reinterpret authoritative
candidate sets, objective contracts, solution_view requirements, forbidden transformations, or complexity limits. You must stay
inside PlannerOutput.model_scope_assessment. scope_commitment must include planner_scope_digest, selected_modeling_level,
included_constructs, excluded_constructs, and scale_strategy. Do not introduce variables, objectives, or constraints for constructs
listed in forbidden_modeling_constructs. Mention unsupported high-level constructs only as excluded assumptions, not as model
equations. Every variable item must include id, name, symbol, domain, indices, meaning, and a public data_ref or evidence_ref. Never
return placeholder-only variables such as {"id":"V001"}. If derived_public_facts.temporal_action_space.compatible_engagements
exists, use it as the index set because each candidate can carry public sensor, weapon, target, launch_step, hit_step, tracking
interval, and effectiveness facts. Do not invent hidden timing or probability fields. If only hit_step, tracking_start_step, and
tracking_end_step are public, express weapon_min_launch_interval_steps and sensor capacity with those public timing fields and let
Builder implement the executable linear constraints. If Planner marks trajectory_kinematic, stochastic_or_robust, or independent
launch timing unsupported, do not use those constructs even if they are natural in the real-world defense domain. The correct model
is the strongest public-data-supported executable formulation, not the most detailed imagined domain model. For objectives and
constraints, include formula_text or description that explicitly refers to the declared variables and public data refs.
solver_capability_notes must be a JSON object. If ROUTER_FEEDBACK is present, repair only the listed ModelDesignCard contract
defects. If MEMORY_CARD is present, use it only as public-safe guidance for modeling choices, recurring omissions, scale-control
lessons, and failed repair patterns. The current public input and PlannerOutput remain the source of instance facts.
```

Appendix C.3. Critic prompt

```
You are the WTA-MAFA v0.4 Critic Agent. Return exactly one JSON object matching CriticReport:
public_input_digest, design_card_digest, passed, defects, checked_dimensions, notes. Check only execution-relevant modeling defects,
public data binding risks, family field mismatches, boundary violations, and build-blocking semantic issues. Do not repair the
design. Do not perform hidden benchmark scoring or external leaderboard evaluation. Also check scope_alignment.
PlannerOutput.model_scope_assessment is binding for the ModelDesignCard. If the design introduces a variable, objective, or
constraint for a construct listed in forbidden_modeling_constructs, return UNSUPPORTED_MODELING_CONSTRUCT with owner "modeler" and
phase "pre_build_quality_gate". If scope_commitment is missing or its planner_scope_digest/selected_modeling_level does not match
PlannerOutput, return SCOPE_COMMITMENT_MISSING, SCOPE_COMMITMENT_DIGEST_MISMATCH, or MODELER_SCOPE_LEVEL_MISMATCH. Each defect must
use this canonical schema:
owner, phase, error_code, message, severity, component_id, path, details. Use owner "modeler" and phase "pre_build_quality_gate" for
model-design defects. severity must be one of minor, warning, major, critical. details must be a JSON object, not a string. If you
need free text, put it in details.text. Prefer stable error_code values when applicable: MISSING_VARIABLE_DOMAIN,
VARIABLE_DECLARATION_MISMATCH, MODEL_VARIABLE_SCHEMA_INSUFFICIENT, MISSING_TEMPORAL_DECISION_VARIABLES, CONSTRAINT_UNENFORCEABLE,
UNBOUND_PUBLIC_DATA_REF, FAMILY_FIELD_MISMATCH, SCOPE_COMMITMENT_MISSING, SCOPE_COMMITMENT_DIGEST_MISMATCH,
MODELER_SCOPE_LEVEL_MISMATCH, UNSUPPORTED_MODELING_CONSTRUCT, NON_PUBLIC_BOUNDARY_VIOLATION. Do not use the planner digest as
design_card_digest. If the exact design_card digest is not visible, still return the required field; the runtime will replace it
with the actual digest before validation. Remember that ModelDesignCard is not the executable authority. Do not reject a design
merely because it does not spell out every Gurobi linearization detail; Builder owns executable implementation. Block only
execution-relevant semantic defects such as missing declared variables, missing public constraint intent, non-public data use, or
impossible public data binding. If candidates expose public hit_step, tracking_start_step, and tracking_end_step, it is acceptable
for the design card to state weapon_min_launch_interval_steps as a public hit_step/timing sliding-window constraint and let Builder
implement it. Do not report missing launch_time variables unless launch_step is public and required by the prompt.
```

Appendix C.4. Error Router prompt

```
You are the WTA-MAFA v0.4 ErrorRouter Agent. Classify defects and route them to planner, modeler, critic, revisor, builder/coder,
solver_agent, tooling, stop, or rollback_elite. Do not edit the model directly.
```

Table C.1
Agent prompt and output-contract index.

Agent	LLM call	Input	Output
Planner	Yes	Public scenario, public instance data, execution-contract summary	Problem family, public fields, candidate objectives and constraints, forbidden data boundary, modeling-scope judgement
Modeler	Yes	Planner output, public input, execution contract	Design Card: variables, objectives, constraints, assumptions, public-data references, scope commitment
Critic	Yes	Design Card and Planner scope	Execution-blocking defects, public-data binding risks, boundary violations, pass/fail
Error Router	Yes	A normalized defect from any stage	Route target, responsible component, retry permission, stop-or-rollback decision
Executable Builder	Yes	Public input, Builder hints, hard execution contract, front-end design summary	Executable Python/Gurobi code, `MODEL_REGISTRY`, `solution_view`

Appendix C.5. Executable Builder prompt

```
You are the Executable Builder Agent in WTA-MAFA Agent v0.4. Your job is to build the authoritative executable optimization model
directly from public input. P3 PlannerOutput, ModelDesignCard, CriticReport, and RevisionPatch are guidance only. They are not
solver-bound authority. Your returned source_code plus literal MODEL_REGISTRY becomes the candidate authority only after
deterministic verification passes. Rules:

- Return exactly one JSON object: {"source_code": "..."}.
- The source_code must define top-level literal MODEL_REGISTRY.
- The source_code must define top-level def build_model(data: dict, config: dict) -> dict.
- The source_code must define top-level def solution_view(model): exactly.
- build_model must construct and return a dictionary with model, variables, metrics, and solution_view.
- The returned dictionary's solution_view value must be the top-level solution_view function object directly. Never return
solution_view as the whole build_model result. Do not return lambda m: solution_view(m), wrappers, partials, closures, or any
callable whose single required argument is not named model.
- build_model must not call optimize(); SolverAgent owns optimize.
- SolverAgent owns optimize only. The builder owns the executable objective.
- Before returning, the builder must set a real decision-variable objective by calling model.setObjective(...) or
model.setObjectiveN(...). Never set a zero placeholder objective.
- Do not store objectives in model._objectives and expect SolverAgent to override them. If there are multiple public objective ids,
keep them in MODEL_REGISTRY.active_objectives/reporting; optimize and replay only the solver-bound objective_contract unless the
public input supplies a supported scalarization contract or supported Gurobi multi-objective contract.
- The generated builder source is the model authority. MODEL_REGISTRY is the human-readable registry used by ModelReporter.
- MODEL_REGISTRY must include schema_version, model_authority, benchmark, sample_id, public_input_digest, builder_entrypoint,
builder_kind, builder_code_digest, variables, objectives, constraints, data_refs, modeling_assumptions, boundary, and
reporting_notes.
- MODEL_REGISTRY must include objective_contract. This is a literal dictionary that declares the exact solver objective being
optimized and replayed.
- MODEL_REGISTRY.objective_contract.schema_version must equal the provided OBJECTIVE_CONTRACT_SCHEMA_VERSION.
- MODEL_REGISTRY.objective_contract must include objective_id, sense, unit, solver_objective_formula, replay_formula, constant_term,
scale_factor, comparison_tolerance, term_schema, and public_data_refs when applicable.
- The Gurobi objective, MODEL_REGISTRY.objective_contract, model._wta_mafa_solution_registry semantic entries,
solution_view.objective_breakdown, and solution_view.replayed_objective_value must all describe the same objective. Do not optimize
one formula and report another.
- If solution_view or public replay includes a runtime objective constant such as total_target_value, include that same constant in
model.setObjective(...). Omitting constants from optimization is only allowed if the declared contract and every replay field also
omit them.
- solution_view(model) must return objective_contract_id, objective_breakdown, and replayed_objective_value in addition to
selected_engagements. objective_breakdown must be a dictionary.
- Each selected_engagements item should include objective_contribution when the declared replay_formula needs selected-term replay.
- MODEL_REGISTRY.schema_version must equal the provided REGISTRY_SCHEMA_VERSION.
- MODEL_REGISTRY.public_input_digest must equal the provided PUBLIC_INPUT_DIGEST.
- MODEL_REGISTRY.builder_code_digest may be an empty string; the runner will bind the final code digest externally.
- MODEL_REGISTRY.boundary must include hidden_reference_used=false, non_public_builder_used=false, and solver_owns_optimize=true.
- Do not include forbidden source tokens for hidden solutions, answer keys, reference-model wording, or non-public builders.
- Do not import os, sys, pathlib, glob, pickle, shutil, subprocess, socket, requests, urllib, or builtins.
- Do not call open(), eval(), exec(), compile(), input(), optimize(), or minimize().
- Use only runtime data paths shown in PUBLIC_INPUT_PROMPT_VIEW and EXECUTABLE_DATA_MANIFEST_PROMPT_VIEW.
- Use P3_FRONTEND_CONTEXT to align variables, objectives, constraints, assumptions, and public data refs with the multi-agent front
end, but do not copy any hidden/reference/trusted wording into source_code.
- MODEL_SCOPE_ASSESSMENT and MODEL_SCOPE_COMMITMENT are binding. Do not upgrade selected_modeling_level. Do not introduce variables
or constraints for forbidden_modeling_constructs. If a domain-specific rule below conflicts with Planner-approved scope, obey the
scope and implement the strongest public-data-supported model inside that boundary.
- If MEMORY_CARD is present, use it only as public-safe guidance about recurring defects, successful formulation patterns,
scale-control lessons, and failed repair patterns. It does not override the current public input, verifier contract, or
BuilderRepairBrief.
- If ROUTER_FEEDBACK targets builder, repair only the listed builder/verifier/post-solver quality defects. Do not change
PUBLIC_INPUT_DIGEST, do not use hidden/trusted/reference code, and do not call optimize().
- If BUILDER_REPAIR_BRIEF.repair_mode is correctness_repair, eliminate the listed hard defects first; do not trade correctness for
scale reduction.
- If BUILDER_REPAIR_BRIEF.repair_mode is formulation_rebuild, preserve all BUILDER_REPAIR_BRIEF.must_preserve semantics while
improving formulation quality targets such as sparse candidate variables, fewer redundant constraints, lower constraints_per_binary,
better MIP gap, or cleaner solution_view replay.
- For formulation_rebuild, improve formulation quality inside MODEL_SCOPE_COMMITMENT only; never use quality feedback as permission
to add unsupported launch-time, trajectory, stochastic, or hidden/reference constructs.
- For formulation_rebuild, do not merely raise solver time limits or loosen constraints. Improve candidate construction, constraint
aggregation, variable indexing, and objective/constraint sparsity using only public data.
- For any repair mode, acceptance requires objective replay consistency, public feasibility when public oracle fields exist, valid
solution_view, and no non-public/trusted/reference leakage.
- PUBLIC_INPUT_PROMPT_VIEW may summarize large lists. It preserves runtime paths, field names, counts, and samples; the actual
build_model(data, config) receives the full public payload.
- BUILDER_EXECUTION_HINTS is a deterministic public-only runtime normalization of the current sample. It is not a selected solution
and does not contain evaluator feedback, hidden references, trusted builders, or answer keys.
```

Table C.2
Defect taxonomy used by the Error Router. Each class maps to a responsible owner and a bounded repair action, and every repair is re-validated before the next round.

Defect class	Observed signals	Owner	Repair action
Design	Unclear public boundary, missing variable, objective drift, incomplete Design Card	Design Layer	Update the Design Card
Construction	Data-binding error, constraint conflict, code-assembly failure, semantic drift	Responsible Builder sub-agent	Local rewrite through the SCW
Solving and decoding	Solver not executable, unstable result, replay inconsistency, export misalignment	Solve or decode stage	Interface and consistency correction
Internal-evaluation threshold	Insufficient executability, objective inconsistency, infeasible decoded plan, or insufficient domain-task quality	Assigned by diagnosis	Targeted repair, or stop and record

```
- BENCHMARK_EXECUTION_CONTRACT is a hard public execution boundary, not a creative-writing hint. Its hard fields fix public data
paths, authoritative candidate sets, objective replay semantics, solution_view schema, mandatory constraint semantics, forbidden
transformations, and complexity envelope.
- Creativity is allowed only inside that contract: choose equivalent formulations, sparse grouping, decomposition, valid cuts,
compact auxiliaries, and solver parameters, as long as the contract still replays and validates.
- If BENCHMARK_EXECUTION_CONTRACT_VALIDATION_REPORT.success is false, do not pretend the sample is solved; surface the contract
defect through a build/registry failure rather than inventing missing data.
- If BENCHMARK_EXECUTION_CONTRACT conflicts with P3_FRONTEND_CONTEXT, MEMORY_CARD, KNOWLEDGE_CARD, or ROUTER_FEEDBACK, obey the hard
fields in BENCHMARK_EXECUTION_CONTRACT.
- When BUILDER_EXECUTION_HINTS.candidate_source.path is present, build decision variables from that runtime path unless
MODEL_SCOPE_COMMITMENT explicitly forbids the modeling level.
- When BUILDER_EXECUTION_HINTS.objective_contract is present, copy it exactly into MODEL_REGISTRY.objective_contract and make the
Gurobi objective plus solution_view replay match it. Do not invent custom or unsupported replay_formula values.
- If build_model needs runtime objective constants, weights, or replay metadata, store them inside
model._wta_mafa_solution_registry. Do not mutate the literal MODEL_REGISTRY.objective_contract inside build_model.
- Implement BUILDER_EXECUTION_HINTS.constraint_templates whenever the listed required public fields exist in the runtime payload.
- For any repeated-event separation rule with an upper bound, treat the upper bound as a hard span constraint, not a scan-stop hint.
Prefer a grouped window-cover formulation: selected events in one repeated-event group must all be covered by one allowed hit_step
window of width floor(upper_s / step_s), with at most one chosen window per group. Use sliding-window cardinality constraints for
lower-bound separation, and only local close-pair conflicts for same-resource/same-radar threshold rules. Never write `if diff >
upper: break` or `continue` as enforcement; farther pairs are also upper-bound conflicts.
- The builder must attach model._wta_mafa_solution_registry with {"family": string, "assignment_var_items": [{"var": solver_var,
"semantic": {...}}]}.
- solution_view(model) must read only model._wta_mafa_solution_registry to recover selected assignments.
- If solution_view needs objective constants, total target value, scalarization weights, replay formula metadata, or selected-term
contribution sums, build_model must store those public-safe replay fields inside model._wta_mafa_solution_registry. Do not create or
read ad hoc model attributes such as model._total_target_value, model._objective_terms, model._weights, or model._data inside
solution_view.
- A valid model._wta_mafa_solution_registry may include objective_contract_id, objective_constant_term, objective_scale_factor,
objective_sense, objective_breakdown_metadata, or other public-safe replay metadata in addition to assignment_var_items.
- Keep the model simple, executable, and public-data grounded. Prefer a robust public candidate assignment MILP over a complex
physical model that needs absent raw fields.

Return JSON only. No Markdown.
```

Appendix C.6. Defect taxonomy and repair ownership

Table C.2 states the taxonomy the Error Router routes on, as referenced in Section 3.5. Each class is defined by the evidence that exposes it, so a defect is attributed before any repair is attempted: a design defect is visible in the Design Card, a construction defect in the SCW state or the assembled code, a solving or decoding defect in the solver and registry interfaces, and a threshold defect in the Internal Evaluation diagnostics. For every routed defect the system records a signature, an owner, the routing history, the remaining budget, and the stopping condition, and every repair is re-validated before the next round.

Appendix C.7. SCW construction algorithm

Algorithm C.1 states the construction loop of Section 3.3 of the main paper: the workspace is initialized deterministically, the Builder Team fills the slots, a deterministic gate checks them against the shared candidate sets and contracts, and a defect produces a slot-scoped repair ticket within a per-owner attempt limit.

Algorithm C.1 SCW-coordinated construction with coherence gating and slot-scoped repair

Require: approved Design Card D, public input I^{pub}, Output Contract κ, per-owner repair limit K
Ensure: executable model C, registry R, mathematical description M; or a retained failure record

Initialize. $E \leftarrow \textsc{InitSCW}(I^{\text{pub}}, \kappa)$: contract interpretation, data-binding plan (candidate sets and supported fields), and variable, objective, constraint, solution-view and assembly slots, each with a fixed owner and the digest of I^{pub}
$\mathcal{T} \leftarrow \emptyset$ {repair tickets}
Construct. $S = \{s_{\text{var}}, s_{\text{obj}}, s_{\text{con}}, s_{\text{view}}\} \leftarrow A_{\text{B}}(\text{View}(E), D, \kappa)$
loop
  Gate. $\mathcal{D} \leftarrow$ schema defects of $S \cup \textsc{Coherence}(S, E)$, which checks that
    variables, objective terms and constraints range over the same bound candidate sets;
    every mandatory constraint is covered and every referenced field is supported by the data binding;
    the solver objective and its replay formula share one source, and the objective is not constant;
    no pairwise $O(|E|^2)$ constraint strategy is used on large candidate sets
  if $\mathcal{D} = \emptyset$ **then**
    Assemble. $(C, R) \leftarrow \textsc{Assemble}(S, E)$; derive M from the same (S, E)
    return (C, R, M)
  end if
  $\tau \leftarrow \textsc{Ticket}(\mathcal{D})$ with owner o_τ, affected slots $A_\tau \subseteq S$, preserved slots $S \setminus A_\tau$
  if $|\{\tau' \in \mathcal{T} : o_{\tau'} = o_\tau\}| \geq K$ **then**
    return failure record $(\mathcal{D}, \mathcal{T}, E)$ {no unbounded retry}
  end if
  $\mathcal{T} \leftarrow \mathcal{T} \cup \{\tau\}$; $A_\tau \leftarrow A_{\text{B}}(\text{View}(E), \tau)$ {regenerate affected slots only}
end loop

Appendix D. Output Contract and Artifact Coverage

A run counts as complete only when every contracted artifact is present. Table D.1 reports the fraction of samples for which each artifact was produced, and Figure D.1 shows the same coverage as a funnel; a formulation alone is not a completed run.

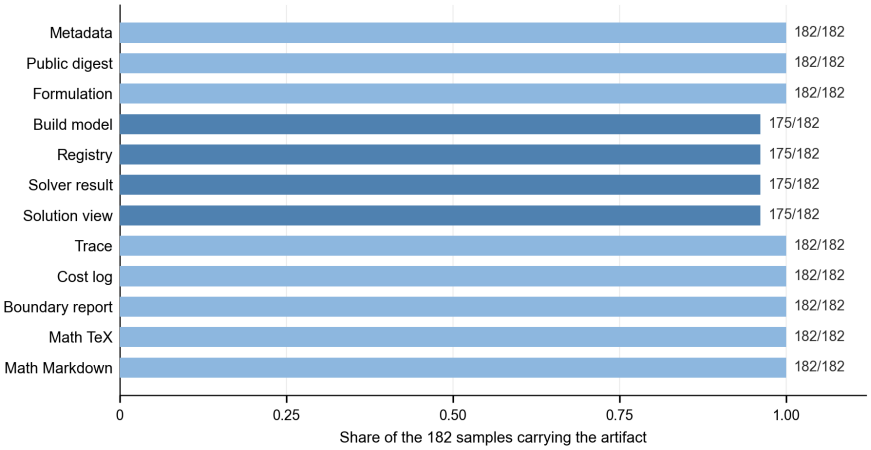


Figure D.1: Artifact coverage funnel across the 182 samples.

Table D.1
Artifact checklist and coverage for WTA-MAFA across the full test sets. Each entry is the fraction of samples carrying that artifact.

Benchmark	N	Metadata	Formulation	Build model	Registry	Solver result	Solution view	Trace
SWTA	162	1.000	1.000	0.981	0.981	0.981	0.981	1.000
MADRA	20	1.000	1.000	0.800	0.800	0.800	0.800	1.000
All	182	1.000	1.000	0.962	0.962	0.962	0.962	1.000

Appendix E. Expanded SWTA and MADRA Results

Appendix E.1. Stage funnels

Tables E.1 and E.2 expand the main-text funnel into its individual stages. The quantity to watch is not the first column but the attrition between columns. For WTA-MAFA the four counts are identical on both benchmarks: every model that builds also solves, decodes and passes public-feasibility validation. For the general systems the counts fall between stages: OR-LLM-Agent loses 13 SWTA samples between solving and a complete output, CoE loses 21 between an executable output and public feasibility, ORPilot builds and solves 144 SWTA models and produces 129 executable outputs, but only 44 of them are publicly feasible, and the OptiMUS adaptation loses samples at both transitions.

On MADRA, OR-LLM-Agent, CoE and OptiMUS v0.3 each reach 0.667, on 4, 2 and 3 samples that are all empty plans, while ORPilot reaches a higher conditional mean than WTA-MAFA, 0.873 on 8 samples against 0.797 on 16. The same reading applies to ORPilot's 0.833 on SWTA. Scalar quality here averages every scored output, and the evaluator applies its time-indexed checks only to plans that carry timing fields; 85 of ORPilot's 129 scored SWTA outputs and 6 of its 8 scored MADRA outputs fail public feasibility once those checks are applied to them, and restricted to feasible outputs its means are 0.806 and 0.792 (Table E.3). We report these values rather than omit them, because they show why a conditional quality score computed on a small or unvalidated set of outputs is not comparable to one computed over broad, validated coverage.

Appendix E.2. Domain-quality dimensions

Table E.3 decomposes the scalar score into the six dimensions the external evaluator computes. Four of them, namely engagement feasibility, resource feasibility, temporal feasibility (not listed in the table) and traceability, are at or near 1.000 for every method, so they carry no discriminative information: any system that produces a decodable plan over the public candidate set satisfies them. The scalar mean is an unweighted average of the dimensions available for each output, which is why it compresses into the narrow 0.600–0.806 band despite large differences between methods. The two dimensions that separate systems are value coverage and high-value coverage, and the main paper scales both against a rescoring of the reference Gurobi solutions, reported on these two dimensions only since the reference solutions are released as assignments without the `hit_step` field that the remaining components read. The main-paper ratios use all 162 cases on the same instances, with a method's failed outputs scored as zero.

The reference solutions optimize the benchmark's lexicographic objective rather than either coverage metric, so they need not bound it. To check this, we maximize each coverage metric directly with Gurobi under the evaluator's public-feasibility checks for SWTA (candidate membership, at most one engagement per target, and weapon inventory). All 324 programs solve to optimality and every maximizer passes the evaluator. The maxima average 0.764 and 0.947 and exceed the reference solution on 151 and 87 of the 162 cases, so the reference solutions are not an upper bound on either metric. These maxima are bounds under the evaluator's checks, which do not enforce time-indexed sensor capacity when a plan carries no timing fields, rather than operational optima.

Two readings of this table should be stated explicitly. First, an empty plan is scored, not discarded: it satisfies every hard constraint vacuously and receives full marks on engagement feasibility, resource feasibility, and traceability, so it reaches 0.529 under the Balanced profile and 0.600 on the unweighted Scalar while covering no target. Every publicly feasible SWTA output of OR-LLM-Agent, CoE, and OptiMUS v0.3 is such a plan, which is why their coverage dimensions are zero. Second, conditioned on each method's own feasible outputs, ORPilot scores above WTA-MAFA on both benchmarks, reaching 0.758 against 0.683 on SWTA and 0.738 against 0.725 on MADRA under the Balanced profile. The benchmark-wide ordering in Table 3 of the main paper therefore rests on how many cases a method closes with a feasible plan, 159 against 44 on SWTA and 16 against 2 on MADRA, rather than on a per-output quality advantage on the cases it does close.

Appendix E.3. Modeling cost

Cost is reported as an operating characteristic, not a ranking criterion. The comparison is not like-for-like: CoE issues a fixed seven calls per sample whether or not the run succeeds, OR-LLM-Agent issues two, ORPilot runs its full workflow on every sample and has the highest mean modeling time at 1128.02 s over 13.87 calls, and the OptiMUS adaptation averages 55.07 calls and 698.75 s because its debugging loop re-enters on failure, despite producing the fewest complete outputs. WTA-MAFA averages 4.87 calls and 397.47 s. The relevant reading is therefore cost *per completed output* rather than per attempt: a method that spends little on samples it never completes is not cheaper in any useful sense. Figure E.1 shows the per-sample relation between LLM calls, modeling time and domain quality.

Appendix E.4. Failure endpoints

Table E.5 lists every failure endpoint per method and benchmark. The contrast is structural rather than quantitative: WTA-MAFA's seven failures are all stopped by gates before a model is assembled (a modeling-risk gate on large samples, one provider

Table E.1
SWTA end-to-end stage funnel. All methods share the backbone, inputs and evaluator.

Method	N	Build	Solve	Executable output	Public feasible	Scalar quality
WTA-MAFA	162	159/162 (98.1%)	159/162 (98.1%)	159/162 (98.1%)	159/162 (98.1%)	0.745
OR-LLM-Agent	162	117/162 (72.2%)	117/162 (72.2%)	104/162 (64.2%)	104/162 (64.2%)	0.600
CoE	162	56/162 (34.6%)	56/162 (34.6%)	56/162 (34.6%)	35/162 (21.6%)	0.620
OptiMUS v0.3	162	98/162 (60.5%)	82/162 (50.6%)	37/162 (22.8%)	31/162 (19.1%)	0.632
ORPilot	162	144/162 (88.9%)	144/162 (88.9%)	129/162 (79.6%)	44/162 (27.2%)	0.833

Table E.2
MADRA end-to-end stage funnel and conditional quality among scored outputs. Conditional quality on MADRA is computed over very different sample counts (16, 4, 2, 3 and 8), so it must be read together with coverage.

Method	N	Build	Solve	Executable	Public feasible	Scored	Scalar quality
WTA-MAFA	20	16/20 (80.0%)	16/20 (80.0%)	16/20 (80.0%)	16/20 (80.0%)	16	0.797
OR-LLM-Agent	20	7/20 (35.0%)	7/20 (35.0%)	4/20 (20.0%)	4/20 (20.0%)	4	0.667
CoE	20	2/20 (10.0%)	2/20 (10.0%)	2/20 (10.0%)	2/20 (10.0%)	2	0.667
OptiMUS v0.3	20	11/20 (55.0%)	8/20 (40.0%)	3/20 (15.0%)	3/20 (15.0%)	3	0.667
ORPilot	20	12/20 (60.0%)	12/20 (60.0%)	8/20 (40.0%)	2/20 (10.0%)	8	0.873

Table E.3
SWTA domain-quality dimensions with common scoring support. Public feasible applies the time-indexed checks described in main-text Table 3 to plans without explicit timing; Non-empty counts the feasible outputs that select at least one engagement. The dimension columns are conditional means over each method's feasible outputs, and Scalar is the unweighted mean of the dimensions available for each output.

Method	Scored	Public feasible	Non-empty	Engage. feas.	Value cov.	Resource feas.	High-value cov.	Traceability	Scalar
WTA-MAFA	159	159/162 (98.1%)	159	1.000	0.126	1.000	0.342	1.000	0.745
OR-LLM-Agent	104	104/162 (64.2%)	0	1.000	0.000	1.000	0.000	1.000	0.600
ORPilot	129	44/162 (27.2%)	44	1.000	0.394	1.000	0.637	1.000	0.806
CoE	39	35/162 (21.6%)	0	1.000	0.000	1.000	0.000	1.000	0.600
OptiMUS v0.3	37	31/162 (19.1%)	0	1.000	0.000	1.000	0.000	1.000	0.600

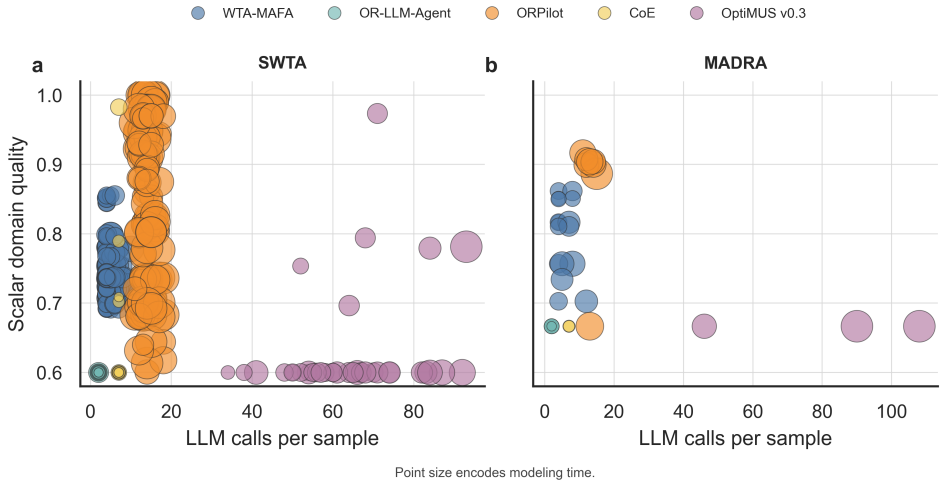


Figure E.1: Per-sample LLM calls versus scalar domain quality on SWTA (a) and MADRA (b); point size encodes modeling time. Each point is one scored output, including outputs that fail public feasibility. Every publicly feasible SWTA output of CoE and OptiMUS v0.3 is an empty plan scoring 0.600 (Table E.3), so their SWTA points above 0.600 are outputs that fail public feasibility; ORPilot's SWTA points include the 85 outputs whose untimed plans fail the time-indexed checks. The MADRA points of OR-LLM-Agent, CoE and OptiMUS v0.3 at 0.667 are empty plans.

or builder contract failure, one unsupported construct, one missing builder slot), whereas the general systems fail across the pipeline: OR-LLM-Agent at the solver, at solution decoding and, on MADRA, at the objective contract, CoE mostly inside its own pipeline before producing an executable output (106/162 on SWTA), ORPilot mostly at public-feasibility validation (85/162 on SWTA, 82 of them because no hit-time assignment satisfies the time-indexed checks), and OptiMUS v0.3 at every stage from construction to public-feasibility validation. Figure E.2 resolves the seven WTA-MAFA failures by phase and owner.

Table E.4
Runtime and interaction cost over the full 182 samples, using uniformly recorded fields.

Method	N	Mean LLM calls	Mean modeling time (s)	Mean total wall time (s)
WTA-MAFA	182	4.87	397.47	398.24
OR-LLM-Agent	182	2.00	190.19	190.19
CoE	182	7.00	139.70	139.70
OptiMUS v0.3	182	55.07	698.75	698.75
ORPilot	182	13.87	1128.02	1155.65

Table E.5
Failure endpoints over the full test sets; each failed sample is counted once. Rates are relative to that method's benchmark denominator. For CoE, 35 of the 106 SWTA and 4 of the 18 MADRA method failures ended in an API connection error.

Benchmark / Method	Failure endpoint	Count	Rate
SWTA / WTA-MAFA	Provider or builder contract	1	0.6%
SWTA / WTA-MAFA	Large-sample modeling-risk gate	1	0.6%
SWTA / WTA-MAFA	Unsupported modeling construct	1	0.6%
MADRA / WTA-MAFA	Large-sample modeling-risk gate	3	15.0%
MADRA / WTA-MAFA	Missing builder slot	1	5.0%
SWTA / OR-LLM-Agent	Solver failure	45	27.8%
SWTA / OR-LLM-Agent	Solution-view failure	13	8.0%
MADRA / OR-LLM-Agent	Solver failure	13	65.0%
MADRA / OR-LLM-Agent	Objective mismatch	3	15.0%
SWTA / CoE	Method failure	106	65.4%
SWTA / CoE	Public-feasibility failure	21	13.0%
MADRA / CoE	Method failure	18	90.0%
SWTA / OptiMUS v0.3	Method failure	38	23.5%
SWTA / OptiMUS v0.3	Objective mismatch	33	20.4%
SWTA / OptiMUS v0.3	Solver failure	31	19.1%
SWTA / OptiMUS v0.3	Timeout	12	7.4%
SWTA / OptiMUS v0.3	Solution-view failure	9	5.6%
SWTA / OptiMUS v0.3	Public-feasibility failure	6	3.7%
SWTA / OptiMUS v0.3	Build failure	2	1.2%
MADRA / OptiMUS v0.3	Method failure	7	35.0%
MADRA / OptiMUS v0.3	Objective mismatch	3	15.0%
MADRA / OptiMUS v0.3	Timeout	3	15.0%
MADRA / OptiMUS v0.3	Solver failure	2	10.0%
MADRA / OptiMUS v0.3	Solution-view failure	2	10.0%
SWTA / ORPilot	Public-feasibility failure	85	52.5%
SWTA / ORPilot	Solution-view failure	17	10.5%
SWTA / ORPilot	Method failure	8	4.9%
SWTA / ORPilot	Timeout	8	4.9%
MADRA / ORPilot	Method failure	6	30.0%
MADRA / ORPilot	Public-feasibility failure	6	30.0%
MADRA / ORPilot	Solution-view failure	4	20.0%
MADRA / ORPilot	Timeout	2	10.0%

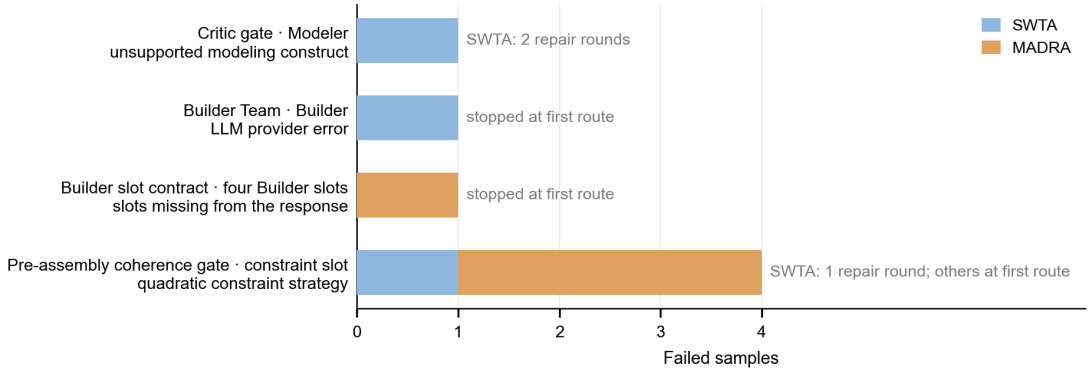


Figure E.2: Where the seven WTA-MAFA failures stop, by gate and responsible component; numbers give the repair rounds completed before the stop.

Appendix F. Expert Reference Comparison

The expert comparison separates two questions that are easy to conflate: how often a generated model can be put side by side with an expert one at all, and how it behaves once it can. Table F.1 answers the first: expert references exist for all 40 representative samples, and WTA-MAFA produces a semantically pairable model for 33 of them. The seven samples without a pairable model are exactly the seven failed runs of the end-to-end evaluation, which stopped before assembling a model; every

Table F.1
Availability and pairing coverage for the expert-reference comparison.

Family	Reference models available	WTA-MAFA models available for pairing	Pairing coverage
SWTA	30/30 (100.0%)	27/30 (90.0%)	27/30
MADRA	10/10 (100.0%)	6/10 (60.0%)	6/10
All representative samples	40/40 (100.0%)	33/40 (82.5%)	33/40

Table F.2
Paired model checks and matched-policy quality, on paired cases only (27 SWTA, 6 MADRA). Quality is the mean target engagement rate; random quality averages 20 feasible draws per pair.

Benchmark	Model source	Gurobi	Greedy feasible	Random feasible	Greedy quality	Random quality
SWTA	Reference model	27/27	27/27	540/540	0.268	0.260
SWTA	WTA-MAFA generated	27/27	27/27	540/540	0.268	0.260
MADRA	Reference model	6/6	6/6	120/120	0.889	0.965
MADRA	WTA-MAFA generated	6/6	6/6	120/120	0.889	0.965

Table F.3
SWTA objectives with each paired model solved as formulated: geometric mean of expert value / WTA-MAFA value per MOP, with the per-case range and the number of paired cases in which the objective is active. Values above 1 favor WTA-MAFA.

MOP	F1 remaining threat value	F2 resource cost	F3 engagement makespan	F4 concentration risk
MOP1	1.00 [1.00, 1.00] (n=5)	0.73 [0.65, 0.90] (n=5)	1.00 [1.00, 1.00] (n=3)	2.00 [2.00, 2.00] (n=1)
MOP2	1.00 [1.00, 1.00] (n=6)	0.70 [0.52, 0.87] (n=6)	1.00 [1.00, 1.00] (n=4)	2.29 [1.50, 3.50] (n=2)
MOP3	0.97 [0.93, 1.00] (n=6)	1.30 [0.80, 1.84] (n=6)	1.00 [1.00, 1.00] (n=4)	2.74 [2.50, 3.00] (n=2)
MOP4	0.32 [0.12, 0.54] (n=5)	5.50 [3.00, 8.60] (n=5)	1.33 [1.00, 2.24] (n=3)	10.07 [7.00, 14.50] (n=2)
MOP5	0.40 [0.18, 0.59] (n=5)	8.23 [4.67, 14.00] (n=5)	2.28 [1.56, 2.84] (n=3)	24.87 [16.50, 37.50] (n=2)

model WTA-MAFA produced on these samples could be paired with its expert reference. Tables F.2–F.4 answer the second, and every statistic in them is computed on the paired cases only, with the same instances on both sides.

Table F.2 reports feasibility and the matched-policy scores. The generated and expert models pass every check alike, and under the common greedy policy and the 20 random feasible policies per pair they obtain identical quality on all 33 paired cases. This identity is expected rather than suspicious: both policies act only on the candidate engagement set a model exposes, and both models read that set from the same public instance data, so the matched policies confirm that the generated model binds the instance correctly but do not rank modeling quality.

What separates the two models is how each is formulated. Table F.3 solves every paired SWTA model as written and reports, per MOP and objective, the geometric mean of expert value / WTA-MAFA value; all four objectives are minimized, so values above 1 favor WTA-MAFA. Each objective enters only the cases in which it is active, so F3 and F4 rest on fewer cases. On MOP1 and MOP2 the generated models coincide with the expert on F1 and F3 in every case, and on MOP3 the largest F1 gap is 7.9%. On MOP4 and MOP5 WTA-MAFA trades a higher remaining threat value for lower resource cost, makespan and concentration risk. The per-case values are the points in the main-text expert comparison figure.

On MADRA every paired generated model is feasible but conservative (Table F.4). In all six generated models the radar channel capacity constraint is grouped by radar only, so it bounds the number of selected engagements over the whole horizon rather than at each time step, and the per-target attempt limit defaults to a single attempt. The resulting plans engage far fewer targets than the expert models and attain a lower aggregate interception probability in every paired case. Together with the four unpaired cases, which are also temporally coupled MADRA instances, this locates the remaining gap to expert modeling in time-indexed resource coupling rather than in feasibility.

Table F.4
MADRA paired cases with each model solved as formulated: selected engagements and aggregate interception probability (higher is better). Both models are publicly feasible in every case.

	Selected engagements		Aggregate interception probability	
Case	Expert	WTA-MAFA	Expert	WTA-MAFA
P1 (single)	10	4	5.40	1.70
P2 (multi3)	20	5	9.90	2.10
P3 (single)	24	6	12.40	2.40
P4 (multi3)	34	6	14.67	2.40
P8 (multi3)	150	10	64.47	4.10
P10 (multi3)	560	20	270.78	8.00

Appendix G. Ablation, Backbone Sensitivity and Robustness

Appendix G.1. Backbone sensitivity

Table G.1 replaces the language model while holding the architecture, prompts, schemas, solver, evaluator and sample slice fixed. End-to-end success moves within a 2.5-point band (80.0–82.5%) and scalar domain quality within 0.007, both far smaller than the 20.0- and 42.5-point drops produced by removing architectural components in the ablation of main-text Figure 6a. Operating cost, by contrast, varies roughly fourfold in modeling time (383.0–1,500.5 s). The contribution therefore tracks the system design rather than the backbone, while cost tracks the provider.

The three Kimi-K3 runtime failures are provider-side and were retried under provider-accommodated settings; they are reported as operating characteristics, not as model-quality measurements.

Appendix G.2. Preference, scale and multi-objective robustness

Robustness is probed along two axes. Table G.2 reweights the common quality dimensions under five preference profiles without changing the generated model, so the spread across profiles reflects what the model actually optimized rather than how it was built: resource-conservative and timeliness profiles score highest (0.792 on SWTA), the effectiveness profile lowest (0.464), consistent with the value-coverage gap identified in the main text (Figure G.1). Table G.3 varies instance scale and interception mode instead, and isolates where the system breaks; Figure 7 in the main paper shows success per multi-objective profile and per MADRA instance family.

Every multi3 case closes, while four single-interception cases fail. At P5, P7 and P9 the pre-assembly coherence gate stops a constraint strategy that grows quadratically with candidate sets of 1,410 to 24,396 engagements, and at P6 the Builder response omits its slots (Appendix J). The multi3 runs of P5, P7 and P9 close on the same candidate sets, so these stops reflect the constraint strategy generated in a run, not the instance size. What the result confirms is that the system withholds a model it cannot build soundly rather than emitting an infeasible plan.



Figure G.1: Preference-profile scores by method; each cell gives the mean and, below it, the number of scored outputs. Dashes mark profiles with no scorable output, and hatched cells rest on fewer than 10 outputs. The Time profile requires timing fields that baseline outputs do not carry; its few ORPilot, CoE and OptiMUS scores of 0 come from outputs that failed the public-feasibility gate. Cells average every scored output: ORPilot's SWTA cells include the 85 outputs whose untimed plans fail the time-indexed checks, and every MADRA output of OR-LLM-Agent, CoE and OptiMUS v0.3 is an empty plan.

Table G.1
Backbone sensitivity under the fixed WTA-MAFA architecture. Decoding temperature is 0.1 for DeepSeek-V4-Pro and Qwen3.7-Max and 1.0 for Kimi-K3.

Backbone	Full success	Scalar quality	Mean LLM calls	Mean modeling time (s)	Runtime failures
DeepSeek-V4-Pro	33/40 (82.5%)	0.7567	4.825	388.4	0
Qwen3.7-Max	33/40 (82.5%)	0.7553	5.025	383.0	0
Kimi-K3	32/40 (80.0%)	0.7619	7.950	1,500.5	3

Table G.2
Preference-profile scores over available outputs. Profiles reweight the common quality dimensions; they do not change the generated model.

Benchmark	Preference profile	Available samples	Mean score
SWTA	Balanced	159	0.683
SWTA	Effectiveness	159	0.464
SWTA	Resource conservative	159	0.792
SWTA	Timeliness	159	0.792
SWTA	High-value protection	159	0.584
MADRA	Balanced	16	0.725
MADRA	Effectiveness	16	0.495
MADRA	Resource conservative	16	0.801
MADRA	Timeliness	16	0.801
MADRA	High-value protection	16	0.681

Table G.3
MADRA scale and interception-mode summary.

Mode	Samples	Full success	Mean modeling time (s)	Interpretation
single	10	6/10 (60.0%)	364.1	Failures concentrated at P5, P6, P7 and P9
multi3	10	10/10 (100.0%)	449.6	All sampled multi3 cases closed the contract

Appendix H. WTA-FormBench: Construction, Boundary and Results

Appendix H.1. Benchmark card

Table H.1 is the frozen specification of WTA-FormBench: what it contains, what an agent may see, and what it deliberately does not measure. Two entries define its scope. The agent-visible input is exactly three files per sample, so recovery is measured against public narrative and data alone. And solver calls are zero by protocol. The benchmark stops at a constructible terminal formulation, because the 18 source families share no common executable interface and forcing one would mix code-adaptation effects into a measurement of formulation recovery.

The four scenario clusters are (i) multi-resource WTA and coverage allocation, (ii) dynamic and spatiotemporal engagement, (iii) sensing, radar and unmanned-system coordination, and (iv) uncertainty, adversarial interaction and replanning. WTA-FormBench runs the full WTA-MAFA workflow but stops once a constructible terminal formulation bundle exists. Because the source papers share no common executable interface or optimization target, adding a solving stage would mix code-adaptation effects into a measurement of formulation recovery.

Appendix H.2. Eighteen source families

Table H.2 lists the 18 families and the formulation pressure each one contributes. They were selected to span four scenario clusters rather than to repeat one problem at different sizes: multi-resource assignment and coverage (P001–P003, P005, P007, P009, P012), dynamic and spatiotemporal engagement (P004, P006, P008, P017), sensing, radar and unmanned-system coordination (P013–P015), and uncertainty, adversarial interaction and replanning (P010, P011, P016, P018).

Appendix H.3. Source references

The 18 source papers define the family semantics. Benchmark samples are not verbatim reproductions of the source experiments: they preserve source-grounded task semantics and formulation relationships while using controlled public instances and frozen contracts.

Appendix H.4. Construction and freeze procedure

Construction separates source-semantic extraction, sample generation, evaluator construction and final freezing. Each step retains its source trace and manifest relations, and no evaluator-private content flows back into agent input.

1. **Source screening.** Select source papers with a clear task scenario, sets and parameters, variables, objectives, hard constraints and traceable data rules.
2. **Semantic extraction.** Extract task objects, decision structure, objective semantics, principal constraints and publishable instance fields.
3. **Family master specification.** For each of P001–P018, fix which structures are source-required semantics and which are optional extensions.

Table H.1
WTA-FormBench benchmark card (frozen specification).

Design item	Frozen specification
Paper families	18, identified as P001–P018
Total samples	144
Splits	dev 54; formal 72; stress 18
Variants per family	S01–S08
Agent-visible input	`scenario_nl.md`, `data_description.md`, `instance_data.json`
Formal target	Recovery of task structure, variables, objectives, constraints and public-data bindings
Solver calls	0 by protocol
Evaluator-private material	Reference formulations and validation traces, withheld from the agent
Source-disjointness	Confirmed in the frozen package
Version	v0.3-frozen-R7D

Table H.2
P001–P018 source-family mapping. Every family contributes formal variants S04–S07 and one stress variant S08.

Family	Source-grounded task family	Main formulation pressure
P001	Sensor-weapon-target assignment	Adaptive large-neighbourhood assignment and public engagement structure
P002	Reliability-aware multi-objective WTA	Reliability and multi-objective target allocation
P003	High-speed vehicle-target assignment	Multiple high-speed vehicles and multiple targets
P004	Dynamic naval engagement planning	Dynamic naval engagement and radar support
P005	Multi-stage WTA	Stage-dependent assignment and resource reuse
P006	Dynamic soft-kill WTA	Suppression, dynamic engagement and temporal coupling
P007	Distributed target allocation	Distributed platforms and cooperative allocation
P008	Multi-solution SWTA	Time-indexed multi-solution engagement decisions
P009	Large-scale multi-objective SWTA	Benchmark-scale assignment and knowledge-driven constraints
P010	Uncertainty-aware WTA	Weapon adequacy uncertainty and kill-threshold logic
P011	Adversarial missile-target assignment	Opponent-aware assignment and scenario-dependent payoff
P012	Radar-launcher resource assignment	Radar-launcher coordination under saturated resources
P013	Cooperative suppressive jamming	Joint task assignment and bandwidth allocation
P014	UAV defense programming	Three-objective UAV defense SWTA
P015	Deployment siting against UAV swarms	Deployment location and UAV-path exposure
P016	Decentralized WTA	Asynchronous communication and local information
P017	Time-driven dynamic WTA	Time sampling and dynamic replanning
P018	Rotational WTA	Launcher orientation, heading error and clustering

4. **Contract normalization.** Map each source formulation onto a common scenario, data and variant contract. Normalization changes symbols and interfaces only; it does not add core semantics the source does not support.
5. **Public sample generation.** Emit `scenario_nl.md`, `data_description.md` and `instance_data.json`, retaining family, variant and source trace per sample.
6. **Canonical reference construction.** Build the evaluator-private reference from the source formulation under a paper-faithful instance rule. This reference never enters agent input.
7. **Controlled variants.** Generate S01–S08 per family: S01–S03 vary the disclosure level, S04–S05 vary scale, S06–S07 add resource, temporal or constraint pressure, and S08 retains a family-specific stress.
8. **Boundary and leakage audit.** Check the agent input list, public and private file boundaries, schema, manifest, source trace and sample hashes. The freeze requires zero public-witness leakage.
9. **Split and batch freeze.** Freeze the dev/formal/stress split, complete independent batch acceptance, then finish assembly, validation-trace completion and manifest integrity checking.

The R7D repair was formal rather than semantic: evaluator-private validation traces were completed for accepted source batches, and a non-self-referential manifest policy was adopted. No public sample, source formulation, reference model, mask, registry, schema or contract content was changed by that repair.

Appendix H.5. Input boundary

Table H.5 specifies what the system may read at each stage. Private material is available only after termination and never re-enters generation for the same sample. Terminal success, that is, the 71/72 and 18/18 rates reported in the main text, is decided by the executable build verifier and the Critic gate under the Output Contract, and does not depend on any quality score.

Appendix H.6. Scenario adaptation across all 18 families

Table 4 in the main paper reports the adaptation for every family. The decision index signature is extracted from the generated Design Card: it lists the public index sets the decision variables actually range over, normalized across the different ways a card may spell an index. Counts are means over the four formal samples of each family.

The signatures track the source task rather than a shared template. Families whose source problem is a plain assignment produce assignment-shaped indices (P001, P010, P016); families with an explicit time dimension index by slot or stage (P004, P006, P012, P018); families built on a sparse candidate set collapse to a single engagement-option index (P008, P009, P014); and the coordination families introduce their own entities: UAVs, far radars and frames for P013, and deployment positions

Table H.3
WTA-FormBench source references, P001–P018.

ID	Citation	Source reference	Venue, year	DOI
P001	Wang et al. [9]	Efficient Adaptive Large Neighborhood Search for Sensor-Weapon-Target Assignment	IEEE TSMC: Systems, 2024	10.1109/TSMC.2024.3431033
P002	Yi et al. [10]	Solving multi-objective weapon-target assignment considering reliability by improved MOEA/D-AM2M	Neurocomputing, 2024	10.1016/j.neucom.2023.126906
P003	Yang et al. [11]	A Vehicle-Target Assignment Framework: Collaborative Missions of Multiple High-Speed Flight Vehicles With Multiple Targets	IEEE TAES, 2025	10.1109/TAES.2025.3595106
P004	Arslan et al. [12]	Solution Approaches for the Dynamic Naval Air Defense Planning Problem	IEEE TSMC: Systems, 2026	10.1109/TSMC.2025.3648686
P005	Chang et al. [13]	Adaptive large neighborhood search algorithm for multi-stage weapon target assignment problem	Computers & Industrial Engineering, 2023	10.1016/j.cie.2023.109303
P006	Tashakori et al. [14]	Dynamic soft-kill weapon-target assignment in naval environments	Computers & Industrial Engineering, 2024	10.1016/j.cie.2024.110606
P007	He et al. [15]	Modelling and simulation of distributed target allocation based on improved NSGA-II in multi-UUV collaborative attack	Applied Soft Computing, 2026	10.1016/j.asoc.2025.114174
P008	Li et al. [16]	Knowledge-guided Competitive Evolutionary Algorithm for Multi-solution Sensor-Weapon-Target Assignment Problem	IEEE TEC, 2026	10.1109/TEVC.2026.3653800
P009	Zheng et al. [1]	Large-Scale Multi-Objective Sensor-Weapon-Target Assignment Optimization: Benchmark Suites and Knowledge-Driven Heuristics	IEEE TEC, 2026	10.1109/TEVC.2026.3683727
P010	Ma et al. [17]	Two-stage hybrid heuristic search algorithm for novel weapon target assignment problems	Computers & Industrial Engineering, 2021	10.1016/j.cie.2021.107717
P011	Luo et al. [18]	Learning-Based Policy Optimization for Adversarial Missile-Target Assignment	IEEE TSMC: Systems, 2022	10.1109/TSMC.2021.3096997
P012	Zheng et al. [19]	Balancing Global and Local Search via Q-Learning in Evolutionary Algorithms for Air Defense Resource Assignment Problems	IEEE TSMC: Systems, 2026	10.1109/TSMC.2026.3690023
P013	Han et al. [20]	A Dynamic UAVs Cooperative Suppressive Jamming Method With Joint Task Assignment and Bandwidth Allocation	IEEE TAES, 2025	10.1109/TAES.2025.3566932
P014	Bai et al. [21]	A hybrid multi-objective evolutionary algorithm with high solving efficiency for UAV defense programming	Swarm and Evolutionary Computation, 2024	10.1016/j.swevo.2024.101572
P015	Li et al. [22]	Optimization of Air Defense System Deployment Against Reconnaissance Drone Swarms	Complex System Modeling and Simulation, 2023	10.23919/CSMS.2023.0003
P016	Hendrickson et al. [23]	Decentralized Weapon-Target Assignment under Asynchronous Communications	J. Guidance, Control, and Dynamics, 2023	10.2514/1.G006532
P017	Liu et al. [24]	A Time-Driven Dynamic Weapon Target Assignment Method	IEEE Access, 2023	10.1109/ACCESS.2023.3332513
P018	Kim et al. [25]	New Weapon Target Assignment Algorithms for Multiple Targets Using a Rotational Strategy and Clustering Approach	IEEE Access, 2022	10.1109/ACCESS.2022.3168718

Table H.4
WTA-FormBench source batches and freeze milestones.

Batch	Families	Function
R1D	P001–P003	Source rebuild and first family contracts
R2E	P004–P006	Dynamic and naval family acceptance
R3C	P007–P009	Distributed, multi-solution and large-scale SWTA acceptance
R4D	P010–P012	Uncertainty, adversarial and radar-launcher resource families
R5D	P013–P015	Jamming, UAV defense and deployment families
R6C	P016–P018	Decentralized, time-driven and rotational families
R7/R7D	All P001–P018	Assembly, boundary audit, manifest check and final freeze

Table H.5
Public and evaluator-private information boundary.

Stage	Accessible	Inaccessible
Agent inference	`scenario_nl.md`, `data_description.md`, `instance_data.json`	Evaluator-private reference material and validation traces
Terminal validation	Agent-generated bundle and public-input digest	Private reference content is never returned to the agent
External evaluation	Public input plus evaluator-private artifacts, after termination	No private evidence re-enters generation for the same sample

and a path-planning graph for P015. Structural size varies with the family as well, from 1.0 to 5.8 decision variables and 2.2 to 10.2 constraints.

P002 is the one family whose Design Card states its index sets in prose that our extractor does not resolve to named public sets; its variable and constraint counts are reported as usual. We report it rather than omit it.

Appendix H.7. Cross-family separation of model vocabulary

To test whether the 18 families really receive different models, we take each family's vocabulary to be the tokens of the names it assigns to its own variables, constraints and objective contract. Free-text descriptions, data-ref blobs and artifact digests are excluded: digests are unique per sample and would manufacture apparent distinctiveness.

Table H.6
Family-distinctive naming. *Unique* counts tokens appearing in this family's variable, constraint and objective names and in no other family.

Fam.	Unique	Representative distinctive tokens
P001	18	select_sensor, sensor_membership_bound, public_weapon_membership
P002	25	area_interceptor_capacity, area_serial_mission_reliability_threshold, independent_attempts_target_kill
P003	9	at_most_one_target_per_hfv, target_max_hfvs_capacity, select_hfv_candidate
P004	10	ship_heading_turn_rate, positive_heading_wrap, radar_support_to_capacity
P005	16	stage_weapon_inventory_capacity, target_damage_threshold_requirement, stage_target_availability
P006	11	jammer_horizontal_vertical_coverage, guidance_compatibility, one_at_a_time_non_jammer
P007	12	platform_weapon_capacity, range_feasibility_restriction, residual_target_threat
P008	2	weapon_min_launch_interval_hit_step_surrogate
P009	21	candidate_engagement_assignment, makespan, operational_risk
P010	11	target_destroyed_only_if_kill_threshold_reached, weapon_unique_assignment, invalid_kill_penalty
P011	7	target_stochastic_kill_probability, missile_exactly_one_target, missile_single_use
P012	22	consecutiveinterceptionspacing, missile_vehicle_load, adrap_expected_intercepted_value
P013	10	uav_radar_bandwidth_allocations, sum_bandwidth_covers_suppressed_radar_working_bandwidth, each_uav_at_most_one_radar_per_frame
P014	5	indexed, weighted, time
P015	30	node_covered_by_selected_deployment, adversarial_path_response, interception_activation_links_to_coverage
P016	26	continuous_engagement_effort, convex_relaxation_effort_objective, binary_weapon_capacity
P017	22	current_missile_storage_selected, missile_engaged_by_some_weapon, missileavailability
P018	17	orientation_heading_feasibility, firing_delay_compliance, launcher_single_target_per_time_slot

Table H.7
WTA-FormBench process and boundary diagnostics. Solver calls are zero by protocol in both splits.

Split	Records	Terminals	LLM calls	Repair attempts	Repair closure	Exceptions
Formal F2	72	71	677	57	57/57	1
Stress F3	18	18	165	13	13/13	0

Over all $\binom{18}{2} = 153$ family pairs the mean Jaccard similarity is 0.065 (median 0.049, 90th percentile 0.136, maximum 0.348). Thirty-five pairs share no naming at all, and no single term is common to all 18 families. A system reusing one template would sit at the opposite end of this scale.

Table H.6 lists, for each family, how many naming tokens occur in that family and in no other, with examples. These are not generic WTA words: they name the mechanism of the source problem: bandwidth coverage for the jamming family, heading wrap and turn rate for the naval family, stage inventory for the multi-stage family, and path coverage for the deployment family.

P008 and P014 are the two families with the fewest distinctive tokens. Both are sparse-candidate families whose decisions collapse to a single engagement-option index, so their naming overlaps heavily with P009; this is a property of the source problems rather than a failure of adaptation.

The per-family counts behind these statistics are those of Table 4 in the main paper, and the family-distinctive tokens are listed in Table H.6.

Appendix H.8. Process and boundary diagnostics

Table H.7 reports what happened during the runs rather than how the outputs scored. The repair columns are the informative ones: 57 repair attempts on the formal split and 13 on the stress split, all of which closed. No sample exhausted its repair budget in either split, so the single formal non-terminal, an archived runner exception on P011-S07, is not a budget failure.

Appendix I. Case Study I: A Repaired SWTA Success

This section follows one run end to end, in the order the system produced it: what each agent was given, what it returned, and how a defect was attributed and repaired. The instance is `MultiSWTA2_Type1_NT50_NS8_NC6_NV8_NW6_O4` (SWTA_MOP1_5, profile MOP2): 50 targets, 8 sensors, 6 channels, 8 vehicles, 6 weapons, four objective variants. It is a repaired success rather than a first-pass one, which is why it was chosen. Table I.1 summarizes the trace, and Figure 8 in the main paper condenses it; the artifacts themselves follow in order.

The defect is the point of the example. An empty assumptions section is not a solver error and would not surface as one; it is a contract violation that the Critic gate catches before construction begins. The Error Router attributes it to Modeler, the owner of that field, and issues a local repair rather than regenerating the pipeline, so the Planner output and public bindings already established are preserved.

Appendix I.1. Public input: the natural-language scenario

The agents read the scenario below. This is the complete text they were given, up to the point of elision; no structured hint about the intended formulation is included.

```
# SWTA MOP1-5 Public Problem Description

## Scope
```

Table I.1
Case I stage trace. The run required one bounded repair round.

Stage	State	Evidence
Input	Public package received	`SWTA_MOP1_5 / NT50_NS8_NC6_NV8_NW6 / MOP2`
Planner	completed	Problem family, public fields and objectives F1-F4 identified
Modeler r0	**contract defect**	`MODEL_DESIGN_SECTION_EMPTY`: the first Design Card gave its assumptions as a dict rather than the required list
Error Router r1	routed to Modeler	`model_design_contract_error` detected in `model_design_contract`; bounded local repair issued
Modeler repair	completed	Bounded local repair of the Design Card only
Critic	passed	No execution-blocking defect remaining
Builder Team	completed	Variables, objectives and constraints assembled through the SCW
Solver	optimal	Gurobi returned within the 300 s limit
Decoding	consistent	Solver value, decoded view and public replay agree
Export	completed	Registry, solution view, audit log and mathematical description emitted

```
This document states the public modeling task shared by the SWTA MOP1-5 benchmark suite. It is a natural-language task
specification, not a reference formulation. It intentionally omits equations, decision-variable encodings, author algorithms, solver
implementations, reference solutions, and evaluator outputs.

## Operational Setting

A weapon-target assignment system must coordinate a finite set of sensors and weapon vehicles against multiple incoming targets.
Targets follow predictable trajectories over a finite engagement horizon. Sensors provide target detection and tracking, while
weapon vehicles conduct interceptions. The benchmark considers both centralized target arrivals at one protected location and
distributed target arrivals at multiple protected locations.

The public benchmark assumptions are:

- the engagement takes place in a two-dimensional plane;
- targets and interceptor flights use uniform linear-motion assumptions within the planning horizon;
- sensor and weapon-vehicle locations remain fixed during an engagement;
- resource availability, trajectories, coverage, timing, and effectiveness data are supplied by each instance;
- a target may be left unengaged when no admissible or selected interception is available.

## Required Interception Plan

For every target, construct either no engagement or one internally consistent engagement that identifies a sensor, a weapon vehicle,
and a weapon launch time. The launch time is the plan decision; the resulting hit time is a distinct, derived event. A selected
engagement must use a sensor-weapon-target combination that remains operationally compatible from the required tracking interval
through the resulting hit time.

## Launch-to-Hit Timing Semantics

The following timing semantics apply to all five MOP variants:

- an interceptor starts at its selected weapon vehicle's fixed position at the selected launch time;
- from the launch time onward, its cumulative travel capability advances according to that vehicle's public weapon-speed value and
the instance's discrete time steps;
- the resulting hit time is the earliest future target-trajectory step at which the interceptor's cumulative travel capability can
reach the target position stored for that step;
- if no such future step exists before the target's valid trajectory ends, the selected engagement is not kinematically reachable;
- the target flight-window check, pre-hit sensor tracking interval, sensor-channel occupancy, detection effectiveness, and
interception effectiveness are evaluated using the resulting hit time where applicable;
- weapon inventory usage and minimum separation between repeated firings are attached to launch events, not silently shifted to the
later hit events.

These statements define public operational semantics only. They do not prescribe decision-variable encodings, equations,
linearizations, or a solver implementation.

## Geometry and Effectiveness Data Semantics

Kinematic reachability and interception effectiveness are related but distinct public concepts:

- the stored target trajectory is the authoritative target position at each discrete step; start/end coordinates may summarize a
trajectory but must not replace the supplied stepwise trace;
- weapon position, weapon speed, launch time, the future target trace, and elapsed discrete steps determine whether and when an
interceptor can physically reach a target;
- the supplied detection-probability and interception-probability arrays are authoritative effectiveness coefficients for the MOP
variant that generated them;
- in spatially dependent variants, those probability arrays already encode the corresponding author-defined spatial effectiveness
calculation at each stored target-trajectory step;
- the effectiveness coefficients are sampled at the applicable tracking or resulting-hit steps and retained when evaluating expected
remaining target value;
- a zero effectiveness coefficient means that the selected resource provides no expected effect at that step. It does not replace,
prove, or disprove the separate launch-to-hit travel calculation;
- therefore a formulation must neither use a positive probability as a substitute for interceptor travel-time feasibility nor
discard the probability arrays from an objective that explicitly requires detection/interception effectiveness.

These distinctions expose public input semantics, not a reference equation or decision-variable encoding.

## Objective Requirements

The benchmark defines four conflicting performance criteria. Preserve them as separate raw objectives unless an experiment
explicitly declares a subset or a documented scalarization.

1. **Expected remaining target value - minimize.** Prefer plans that reduce the value expected to remain after accounting for target
importance and the supplied sensor-detection and weapon-interception effectiveness coefficients at the applicable resulting-hit
step.
```

```
2. **Resource usage cost - minimize.** Minimize the total operating cost of the sensors and weapon vehicles assigned to engagements.
An unengaged target contributes no resource usage cost.
3. **Launch makespan - minimize.** Complete the planned launch scheme as early as possible by minimizing the latest selected weapon
launch time. This objective does not redefine launch time as hit time; every derived hit must still remain inside its target's
flight window.
4. **Operational concentration risk - minimize.** Avoid relying too heavily on any single sensor or weapon vehicle. Balance
engagements across available resources so that the loss or overload of one resource does not compromise many targets.

These objectives are not interchangeable: a high-effectiveness plan may cost more, a low-cost plan may be slower, and concentrating
assignments on the strongest resources may increase operational risk.
```

Appendix I.2. Public input: instance structure

The instance file is 1.5 MB of arrays. Its structure, with values elided, is:


```
{
  "adapter_provenance": {
    "adapter": "public_input_compiler.swta_v2",
    "compile_summary": {
      "array_names": [
        "detection_probability",
        "... (26 more entries omitted)"
      ],
      "counts": {
        "sensor_classes": 6,
        "sensors": 8,
        "targets": 50,
        "weapon_classes": 6,
        "weapon_vehicles": 8
      },
      "detection_probability_scale": 1.0,
      "detection_probability_shape": [
        8,
        "... (2 more entries omitted)"
      ],
      "engagement_option_count": 3200,
      "interception_probability_scale": 1.0,
      "interception_probability_shape": [
        8,
        "... (2 more entries omitted)"
      ],
      "positive_detection_pair_count": 400,
      "positive_interception_pair_count": 400,
      "required_executable_fields": [
        "sets",
        "... (15 more entries omitted)"
      ],
      "static_candidate_count": 3200
    },
    "derivation_boundary": "Fields are deterministic transformations of frozen public SWTA
        arrays, CSV views, objectiv ...",
    "derived_fields": [
      "sets",
      "... (15 more entries omitted)"
    ],
    "engagement_option_fields": [
      "sensor",
      "... (12 more entries omitted)"
    ],
    "external_evaluator_result_used": false,
    "hidden_or_reference_artifacts_used": false,
    "no_downsampling": true,
    "objective_ids": [
      "F1",
      "... (3 more entries omitted)"
    ],
    "schema_version": "wta_mafa_agent_v0_4_clean.public_input_compiler.v1",
    "source_file_checks": {
      "checked_file_count": 9,
      "checks": [
        {
          "exists": true,
          "expected_sha256": "e52d83ae8fb59c79441e8a5463d17177aa025bcbd8e1e9594d7fa87ff6298339",
          "observed_sha256": "e52d83ae8fb59c79441e8a5463d17177aa025bcbd8e1e9594d7fa87ff6298339",
          "ok": true
        },
        "... (8 more entries omitted)"
      ],
      "failed_file_count": 0
... (577 further lines omitted)
```


Appendix I.3. Planner output


```
{
  "artifact_type": "planner_output",
  "benchmark": "SWTA_MOP1_5",
  "constraint_candidates": [
    {
      "id": "target_at_most_one",
      "implementation_freedom": "grouping or equivalent sparse aggregation allowed",
```

```
      "pattern_id": "assignment_partition_limit",
      "required_data_refs": [
        "derived_public_facts.temporal_action_space.compatible_engagements"
      ],
      "scope": "target",
      "semantics": "for every target t, sum_{e in E: target[e]=t} x[e] <= 1"
    },
    {
      "id": "sensor_channel_capacity_time_indexed_weighted",
      "implementation_freedom": "interval sweeps or sparse time buckets allowed; unweighted
          count is forbidden",
      "pattern_id": "time_indexed_resource_capacity",
      "required_data_refs": [
        "derived_public_facts.temporal_action_space.compatible_engagements",
        "public_instance_data.sensor_channel_capacity",
        "... (1 more entries omitted)"
      ],
      "scope": "sensor,time",
      "semantics": "for each sensor s and public time step tau, sum over selected engagements
          active at sensor s and time tau of target_channel_need[target[e]] * x[e] <=
          sensor_channel_capacity[s]"
    },
    "... (2 more entries omitted)"
  ],
  "executable_risks": [
    {},
    {},
    "... (4 more entries omitted)"
  ],
  "forbidden_data_boundaries": [
    "hidden_reference_or_trusted_artifacts_available_to_agent",
    "external_evaluator_result_used or evaluator_only_assets",
    "... (6 more entries omitted)"
  ],
  "model_scope_assessment": {
    "assessment_digest": "cd04fc7da5a8f7c21963e5745a4790bc456ad4babf5949700d7e554fd0a2f94c",
    "assessment_note": "PUBLIC_MODEL_SCOPE_DRAFT floor suggested trajectory_kinematic, but the
        executable data manifest shows trajectory/position/velocity fields are
        described_but_not_materialized and are coder_forbidden. BENCHMARK_EXECUTION_CONTRACT
        also forbids reconstructing hidden launch-time or trajectory variables. T ...",
    "builder_directives": [
      "Implement only the Planner-approved modeling level:
          time_indexed_sparse_public_candidate_assignment.",
      "Use sparse public candidate variables x[e] over
          derived_public_facts.temporal_action_space.compatible_engagements.",
      "... (6 more entries omitted)"
    ],
    "forbidden_modeling_constructs": [
      {
        "blocked_terms": [
          "launch_time",
          "launch time decision",
          "... (1 more entries omitted)"
... (167 further lines omitted)
```


Appendix I.4. Model Design Card


```
{
  "artifact_type": "model_design_card",
  "assumptions": [
    {},
    {},
    "... (7 more entries omitted)"
  ],
  "benchmark": "SWTA_MOP1_5",
  "benchmark_contract_bindings": {
    "binding_digest": "ff3b82242fe906524ba45261161cda1d29302314c30e3efc8e0574417c818b16",
    "candidate_sets": [
      {
        "authoritative": true,
        "binding_source": "computed_from_model_design_card",
        "bound": true,
        "filtering_allowed": false,
        "id": "E",
        "required_item_fields": [
          "sensor",
          "weapon_vehicle",
          "... (6 more entries omitted)"
        ]
      }
    ],
    "constraint_contracts": [
      {
        "binding_source": "computed_from_model_design_card",
        "bound_data_refs": [
          "derived_public_facts.temporal_action_space.compatible_engagements"
        ],
        "constraint_id_bound": true,
        "id": "target_at_most_one",
        "pattern_id": "assignment_partition_limit",
        "required_data_refs": [
          "derived_public_facts.temporal_action_space.compatible_engagements"
```

```
        ]
      },
      {
        "binding_source": "computed_from_model_design_card",
        "bound_data_refs": [
          "derived_public_facts.temporal_action_space.compatible_engagements",
          "public_instance_data.sensor_channel_capacity",
          "... (1 more entries omitted)"
        ],
        "constraint_id_bound": true,
        "id": "sensor_channel_capacity_time_indexed_weighted",
        "pattern_id": "time_indexed_resource_capacity",
        "required_data_refs": [
          "derived_public_facts.temporal_action_space.compatible_engagements",
          "public_instance_data.sensor_channel_capacity",
          "... (1 more entries omitted)"
        ]
      },
      "... (2 more entries omitted)"
    ],
    "contract_digest": "58f83c5267630029036c3015bac20e5ddad2e417abb22a9e1335cf510dbdcd00",
    "contract_id": "bec.swta.MultiSWTA2_Type1_NT50_NS8_NC6_NV8_NW6_O4.v1",
    "contract_projection": {
      "active_objectives": {
        "all_available_objective_ids": [
          "F1",
          "F2",
          "... (2 more entries omitted)"
        ],
        "objective_count": 4,
        "objective_definitions": {
          "F1": {
            "description": "Expected remaining target value after selected defense effects.",
            "evidence_ref": "public_problem_description",
            "formula_text": "Minimize expected remaining target value after selected sensor-
                weapon-target engagements, using public detection and interception
                effectiveness.",
            "id": "F1",
            "name": "expected_remaining_target_value",
            "sense": "minimize",
            "uses": [
              "target_value",
              "engagement_options",
              "... (1 more entries omitted)"
            ]
... (447 further lines omitted)
```


Appendix I.5. Critic report


```
{
  "artifact_type": "critic_report",
  "checked_dimensions": [
    "executability",
    "public_data_fidelity",
    "objective_presence",
    "constraint_coverage",
    "family_field_alignment",
    "scope_alignment",
    "non_public_boundary"
  ],
  "defects": [],
  "design_card_digest": "e511328991a6e1e07d8a0fd0f986bbad9b18787edd03ee7eaa84a2ee47aaf2e8",
  "notes": [
    "P3 critic is a pre-build semantic gate, not the external benchmark evaluator.",
    "planner_digest=d8b3cdf8bc15c37b01a2a38982e5947fe665b807c631e230366e8c950aec95a2",
    "memory_card_digest="
  ],
  "passed": true,
  "public_input_digest": "44b6357e874314c9b20730994a86eabc1379aa9d150ba3f53bc32a742eda40bf"
}
```


Appendix I.6. Error Router decision and route history


```
{
  "artifact_type": "error_route_decision",
  "decision_digest": "fc2738f94ce88072ecb18ad1bf6e7c2cf0d09367bf848c01d00fa5797d853cd0",
  "defects": [
    {
      "component_id": "",
      "details": {},
      "error_code": "MODEL_DESIGN_SECTION_EMPTY",
      "message": "assumptions must be a non-empty list.",
      "owner": "modeler",
      "phase": "model_design",
      "severity": "critical"
    }
  ],
  "public_input_digest": "44b6357e874314c9b20730994a86eabc1379aa9d150ba3f53bc32a742eda40bf",
  "reason": "model_design_contract_error detected in model_design_contract; route to modeler for
      bounded local repair.",
  "retry_allowed": true,
```

```
  "rollback_elite": false,
  "round_index": 1,
  "route_to": "modeler",
  "source_phase": "model_design_contract"
}
```


```
[
  {
    "artifact_type": "error_route_decision",
    "decision_digest": "fc2738f94ce88072ecb18ad1bf6e7c2cf0d09367bf848c01d00fa5797d853cd0",
    "defects": [
      {
        "component_id": "",
        "details": {},
        "error_code": "MODEL_DESIGN_SECTION_EMPTY",
        "message": "assumptions must be a non-empty list.",
        "owner": "modeler",
        "phase": "model_design",
        "severity": "critical"
      }
    ],
    "public_input_digest": "44b6357e874314c9b20730994a86eabc1379aa9d150ba3f53bc32a742eda40bf",
    "reason": "model_design_contract_error detected in model_design_contract; route to modeler
        for bounded local repair.",
    "retry_allowed": true,
    "rollback_elite": false,
    "round_index": 1,
    "route_to": "modeler",
    "source_phase": "model_design_contract"
  }
]
```


Appendix I.7. Generated executable model

The Builder Team emits a single Gurobi program. The excerpt below keeps the header and the objective-and-constraint construction.

```
from __future__ import annotations

import gurobipy as gp
from gurobipy import GRB
from wta_mafa_agent_v0_4_clean.execution.solution_view_contract import SOLUTION_REGISTRY_ATTR
from wta_mafa_agent_v0_4_clean.runtime.builder_runtime_helpers import (
    PublicDataView,
    attach_solution_registry,
    runtime_linear_terms,
    runtime_solution_view_from_registry,
)

MODEL_REGISTRY = {'schema_version': 'wta-mafa-executable-first-model-registry-v1',
 'model_authority': 'executable_builder',
 'benchmark': 'SWTA_MOP1_5',
 'sample_id': 'MultiSWTA2_Type1_NT50_NS8_NC6_NV8_NW6_O4',
 'public_input_digest': '44b6357e874314c9b20730994a86eabc1379aa9d150ba3f53bc32a742eda40bf',
 'builder_entrypoint': 'build_model',
 'builder_kind': 'gurobi',
 'builder_code_digest': '',
 'variables': [{'id': 'x_E',
                'symbol': 'x[e]',
                'domain': 'binary',
                'description': '1 if the public candidate e is selected.'}],
 'objectives': [{'id': 'swta.expected_remaining_target_value_plus_cost',
                 'sense': 'minimize',
                 'description': 'Call runtime_linear_terms(data, '
                                'candidates=view.engagement_options(), '
                                "contract=MODEL_REGISTRY['objective_contract'], family=family). "
                                'Use returned constant_term and per-candidate coefficients to set '
                                'Gurobi objective: constant_term + sum(coefficient[e] * x_E[e]).'}],
 'objective_contract': {'schema_version': 'wta_mafa_agent_v0_4.objective_contract.v1',
                        'objective_id': 'swta.expected_remaining_target_value_plus_cost',
                        'sense': 'minimize',
                        'unit': 'scalarized_public_cost',
                        'solver_objective_formula': 'constant total_target_value plus selected '
                                                    'public resource costs minus selected expected '
                                                    'defended value',
                        'replay_formula': 'swta.expected_remaining_target_value_plus_cost',
                        'constant_term': 0.0,

# ... (imports, data binding and helper definitions omitted) ...

    return runtime_solution_view_from_registry(registry)


def build_model(data: dict, config: dict) -> dict:
    family = "swta"
    view = PublicDataView(data)
    candidates = view.engagement_options()
    model = gp.Model(str(MODEL_REGISTRY.get("sample_id", "wta_mafa_model")))
    try:
        model.Params.OutputFlag = 0
```

```
    except Exception:
        pass
    x = [model.addVar(vtype=GRB.BINARY, name=f"x_{i}") for i, _ in enumerate(candidates)]
    terms = runtime_linear_terms(
        data,
        candidates=candidates,
        contract=MODEL_REGISTRY["objective_contract"],
        family=family,
    )
    term_rows = terms.get("terms", [])
    expr = float(terms.get("constant_term") or 0.0)
    for i, var in enumerate(x):
        coeff = 0.0
        if i < len(term_rows) and isinstance(term_rows[i], dict):
            coeff = float(term_rows[i].get("coefficient") or 0.0)
        var.Obj = coeff
        expr = expr + coeff * var
    sense = str(terms.get("sense") or MODEL_REGISTRY["objective_contract"].get("sense") or "").lower()
    model.setObjective(expr, GRB.MINIMIZE if sense == "minimize" else GRB.MAXIMIZE)
    constraint_count = _add_public_constraints(model, x, candidates)
    attach_solution_registry(
        model,
        public_input=data,
        variables=x,
        candidates=candidates,
        terms=terms,
        contract=MODEL_REGISTRY["objective_contract"],
        family=family,
        extra={"builder_slot_digests": ASSEMBLY_SLOT_PLAN.get("slot_digests", {})},
    )

# ... (full file: case_studies/case01_swta_success/builder_code.py) ...
```

Appendix I.8. Model registry

The registry binds solver variables back to task semantics, and is what makes the decoded view replayable by an independent evaluator.


```
{
  "builder_slot_assembly_report": {
    "artifact_type": "builder_slot_assembly_report",
    "assembly_report_digest":
        "485ec5361caabc88b186f021f96b953033376d095378f131383ba274acd61ff8",
    "assembly_shells": [
      "import shell",
      "... (4 more entries omitted)"
    ],
    "builder_code_digest": "bd23b264fdecaf1cfb8f983c80fbc356e105e6af27311d41016857358d9f1842",
    "contract": {
      "contract_digest": "f5ef569d6268c81581ea3bdcf63956ef4c418c17ab929fb166f65a37055e61f3",
      "defects": [],
      "objective_contract": {
        "contract": {
          "comparison_tolerance": {
            "absolute": 1e-06,
            "relative": 1e-06
          },
          "constant_term": 0.0,
          "objective_id": "swta.expected_remaining_target_value_plus_cost",
          "public_data_refs": [
            "public_instance_data.engagement_options",
            "... (1 more entries omitted)"
          ],
          "replay_formula": "swta.expected_remaining_target_value_plus_cost",
          "scale_factor": 1.0,
          "schema_version": "wta_mafa_agent_v0_4.objective_contract.v1",
          "sense": "minimize",
          "solver_objective_formula": "constant total_target_value plus selected public resource
              costs minus selected expected defended value",
          "term_schema": {
            "constant_aggregation": "sum_once_per_target",
            "contribution_field": "objective_contribution",
            "effectiveness_field": "combined_effectiveness",
            "sensor_cost_field": "sensor_cost",
            "target_field": "target",
            "target_value_field": "target_value",
            "weapon_cost_field": "weapon_cost"
          },
          "unit": "scalarized_public_cost"
        },
        "defects": [],
        "objective_contract_digest":
            "f94d0db21807e4e3ee421fb58fa97cc7893f282f2104492bd3163b586521b645",
        "objective_contract_id": "swta.expected_remaining_target_value_plus_cost",
        "schema_version": "wta_mafa_agent_v0_4.objective_contract_validation_report.v1",
        "success": true,
        "validation_report_digest":
            "31b4caf4f6c5482b4c6529b269d531ddf4bd71af7616f9c97debaff1ad40b845"
      },
      "public_input_digest": "44b6357e874314c9b20730994a86eabc1379aa9d150ba3f53bc32a742eda40bf",
      "registry_digest": "5be1bca990f9c8be707f0b88d56fe0254e4ff445d1624798087e4abc8b0bdc24",
```

```
      "registry_schema_version": "wta-mafa-executable-first-model-registry-v1",
      "role": "executable_first_contract",
      "success": true
... (632 further lines omitted)
```


Appendix I.9. Solver result and decoded solution


```
{
  "best_bound": 32.10171954584357,
  "builder_kind": "gurobi",
  "code_digest": "bd23b264fdecaf1cfb8f983c80fbc356e105e6af27311d41016857358d9f1842",
  "constraint_count": 74,
  "defects": [],
  "dry_run_stub": false,
  "gurobi_status_name": "OPTIMAL",
  "incumbent_available": true,
  "mip_gap": 0.0,
  "model_digest": "5be1bca990f9c8be707f0b88d56fe0254e4ff445d1624798087e4abc8b0bdc24",
  "node_count": 1.0,
  "nonzero_count": 12800,
  "objective_value": 32.10171954584358,
  "public_input_digest": "44b6357e874314c9b20730994a86eabc1379aa9d150ba3f53bc32a742eda40bf",
  "raw_status": "2",
  "real_solver_invocation": true,
  "role": "solver_agent",
  "runtime_seconds": 0.01399993896484375,
  "solution_count": 3,
  "solution_view": {
    "objective_breakdown": {
      "constant_term": 43.22681305130985,
      "metadata": {
        "candidate_source_report": {
          "candidate_count": 3200,
          "candidate_source": {
            "candidate_set_id": "E",
            "contract_digest":
                "58f83c5267630029036c3015bac20e5ddad2e417abb22a9e1335cf510dbdcd00",
            "count": 3200,
            "shape": "list",
            "source_kind": "benchmark_execution_contract"
          },
          "family": "swta",
          "field_names": [
            "sensor",
            "weapon_vehicle",
            "target",
            "hit_step",
            "... (22 more entries omitted)"
          ],
          "public_data_view_digest":
              "e59e7a86088f342bfdb289a7d639e3960438b4f48eeecb9161cf46e84e1971d9",
... (169 further lines omitted)
```



```
{
  "objective_breakdown": {
    "constant_term": 43.22681305130985,
    "metadata": {
      "candidate_source_report": {
        "candidate_count": 3200,
        "candidate_source": {
          "candidate_set_id": "E",
          "contract_digest": "58f83c5267630029036c3015bac20e5ddad2e417abb22a9e1335cf510dbdcd00",
          "count": 3200,
          "shape": "list",
          "source_kind": "benchmark_execution_contract"
        },
        "family": "swta",
        "field_names": [
          "sensor",
          "weapon_vehicle",
          "target",
          "hit_step",
          "... (22 more entries omitted)"
        ],
        "public_data_view_digest":
            "e59e7a86088f342bfdb289a7d639e3960438b4f48eeecb9161cf46e84e1971d9",
        "schema_version": "wta_mafa_agent_v0_4.public_data_view.v1"
      },
      "runtime_helper": "attach_solution_registry"
    },
    "objective_contract_id": "swta.expected_remaining_target_value_plus_cost",
    "objective_sense": "minimize",
    "replayed_objective_value": 32.10171954584357,
    "scale_factor": 1.0,
    "selected_contribution_count": 8,
    "selected_contribution_sum": -11.125093505466273,
    "selected_engagement_count": 8
  },
  "objective_contract_id": "swta.expected_remaining_target_value_plus_cost",
  "replayed_objective_value": 32.10171954584357,
  "selected_engagements": [
```

```
    {
      "best_detection_step": 0,
      "best_interception_step": 0,
      "combined_effectiveness": 1.0,
      "detection_probability": 1.0,
      "detection_probability_max": 1.0,
      "earliest_hit_step": 1,
      "feasible_candidate_count": 48,
      "feasible_hit_intervals": [
        {
... (129 further lines omitted)
```


Appendix I.10. Mathematical description

The mathematical description is derived from the same modeling basis as the executed code, rather than written separately. The artifact below is reproduced as exported, including the digests that tie it to the executed program; its trust status is `scw_executable_verified`, meaning the description was derived from a model that actually built and solved.

Instance: SWTA / MultiSWTA2_Type1_NT50_NS8_NC6_NV8_NW6_O4
Objective contract: `swta.expected_remaining_target_value_plus_cost`
Math trust status: `scw_executable_verified`
Source mode: `scw_executable_contract_swta`

Executable evidence.

Workspace digest	`1cd0e516a67488aa`
Slot digest	`c6ea67785063a59f`
Builder-code digest	`bd23b264fdecaf1c`
Model-registry digest	`5be1bca990f9c8be`
BEC digest	`58f83c5267630029`
External objective-report digest	`31bfc5172256f0f7`
LLM reinterpretation used	`false`
Feedback to the current agent allowed	`false`

SWTA contract-level formulation.

$$
\begin{aligned}
\min_{x} \quad & \sum_{t\in T} v_t + \sum_{e\in E} \left(c_e^S + c_e^W - v_{t(e)} p_e\right) x_e \\
\text{s.t.} \quad & \sum_{e\in E:\, t(e)=t} x_e \le 1 && \forall t \in T \\
& \sum_{e\in E:\, s(e)=s,\ a_e\le\tau\le b_e} q_{t(e)} x_e \le C_s && \forall s \in S,\ \forall \tau \\
& \sum_{e\in E:\, w(e)=w} x_e \le I_w && \forall w \in W \\
& \sum_{e\in E:\, w(e)=w,\ \tau\le h_e\le\tau+d_w-1} x_e \le 1 && \forall w \in W,\ \forall \tau \\
& x_e \in \{0,1\} && \forall e \in E.
\end{aligned}
$$

Symbols. E is the authoritative compatible candidate set; v_t is target value; c_e^S and c_e^W are public sensor and weapon costs; p_e is combined effectiveness; q_t is target channel need; C_s is sensor capacity; I_w is weapon inventory; h_e is hit step; d_w is the minimum launch interval.

Appendix J. Case Study II: A MADRA Failure Stopped at the First Route

The second case is a failure that terminates cleanly. The instance is `MADRA_OPEN_P6_single` (P6, 50 targets, single-interception mode). The front end completed and produced a formulation, but construction failed. The Error Router stopped the run at its first route and retained the partial artifacts, so the run was neither retried indefinitely nor silently discarded. Table J.1 and Figure J.1 summarize the trace.

Two properties matter here. First, the failure is attributed to the slots and phase that produced it rather than reported as a generic exception, which is what makes the failure taxonomy in Table E.5 possible. Second, the run stops: when a defect is classified as non-retryable, the Error Router ends the run at once and records the failure type instead of consuming the budget on repeated attempts. This sample is one of the four MADRA failures behind the 16/20 figure in the main text.

Appendix J.1. Public input: the natural-language scenario

```
# MADRA-LIB Public Problem Description

## Scope And Claim Boundary

This document states the public modeling task used by the MADRA-LIB P1-P10
Paper-Spec Open Reconstruction v1. It is a natural-language specification, not
a reference formulation. It intentionally omits equations, decision-variable
```

Table J.1
Case II stage trace. The run stopped at the first route decision, before any repair, with its failure record retained.

Stage	State	Evidence
Input	Public package received	`MADRA_OPEN_P6_single / P6_single_targets50`
Front end	formulation present	`objective_count=1`; formalization complete but not yet a verified executable model
Builder Team	**slot contract failed**	`BUILDER_SLOT_MISSING`: the Builder response omitted the variable, objective, constraint and solution-view slots
Error Router	stop at round 1	Classified as a non-retryable tool error; the repair ticket was recorded but not executed
Artifact state	partial artifacts retained	`build_model_present=False`; `registry_present=False`; formulation, SCW state and mathematical export manifest retained for audit
Denominator	counted as failure	The sample remains in the 20-case MADRA denominator

```
encodings, author algorithms, solver implementations, reference solutions, and
evaluator outputs. The reconstructed P1-P10 numerical instances follow the
published scale and physical rules but are not the authors' unreleased exact
P1-P10 files.

## Operational Setting

A weapon-target assignment system must coordinate fixed guidance radars and
paired launcher resources against incoming aerial targets over a finite,
discrete-time horizon. Every target follows a known straight-line trajectory
toward the defended region. Target type determines speed and importance.
Radar type determines coverage, field of view, tracking-channel demand, and
guidance accuracy. Launcher type determines inventory, interceptor speed,
engagement geometry, and target-dependent interception effectiveness.

The numerical instance supplies every target trajectory input, all radar and
launcher locations and headings, resource capacities, the time step, and the
physical effectiveness tables. Factorized radar and launcher hit-time
intervals are supplied as public derived geometry data. They identify when an
individual radar can continuously support a target and when an individual
launcher can reach it; a compatible radar-launcher hit opportunity exists only
where the corresponding intervals overlap.

## Required Interception Plan

For each target and each attempt allowed by the selected case protocol, choose
either no engagement or one complete engagement. A complete engagement names
the target, attempt number, launcher, guidance radar, and intended hit step.
An unused attempt consumes no launcher inventory or radar channel capacity.

The `single` protocol permits at most one attempt per target. The `multi3`
protocol permits up to three ordered attempts per target. Used attempt numbers
must be contiguous from the first attempt; a later attempt cannot be used when
an earlier attempt for that target is unused.

## Primary Objective And Reported Metrics

The primary objective is to maximize aggregate interception effectiveness.
For each target, combine the success probabilities of all valid attempts as
independent opportunities and measure the resulting cumulative probability of
intercepting that target. Aggregate effectiveness is the sum of these target
probabilities over the scenario.

The number and rate of targets receiving positive interception probability may
be reported for interpretation. Interception rates broken down by target type
are auxiliary metrics only. They do not replace or alter the primary
aggregate-effectiveness objective.

## Feasibility Requirements

A valid plan must satisfy all applicable requirements simultaneously:

- **Complete assignment:** every used attempt names exactly one known target,
```

Appendix J.2. Public input: instance structure

The structure below keeps the fields the run reads, with values elided.

```
{
  "adapter_provenance": {
    "adapter": "public_input_compiler.madra_v2",
    "compile_summary": {
      "engagement_option_count": 1410,
      "launcher_count": 8,
      "launcher_interval_count": 289,
      "max_attempts_per_target": 1,
      "radar_count": 8,
      "radar_interval_count": 335,
      "required_executable_fields": [
        "sets",
        "... (9 more entries omitted)"
      ],
      "static_candidate_count": 3200,
      "target_count": 50
```

```
    },
    "derivation_boundary": "Fields are deterministic transformations of frozen public MADRA CSV
        files, case config, in ...",
    "derived_fields": [
      "sets",
      "... (9 more entries omitted)"
    ],
    "engagement_option_fields": [
      "target_id",
      "... (10 more entries omitted)"
    ],
    "external_evaluator_result_used": false,
    "hidden_or_reference_artifacts_used": false,
    "no_downsampling": true,
    "objective_ids": [
      "aggregate_interception_probability"
    ],
    "schema_version": "wta_mafa_agent_v0_4_clean.public_input_compiler.v1",
    "source_file_checks": {
      "checked_file_count": 12,
      "checks": [
        {
          "exists": true,
          "expected_sha256": "83b5aeaa8b75c083f32e353363fa3a23662a1edf58c8b7c4ea08117a73d8b701",
          "observed_sha256": "83b5aeaa8b75c083f32e353363fa3a23662a1edf58c8b7c4ea08117a73d8b701",
          "ok": true
        },
        "... (11 more entries omitted)"
      ],
      "failed_file_count": 0
    },
    "source_file_checks_digest":
        "db2772b224a9bb60ffef35089863670f161b39306a0b2a74fbe14417d40d2373",
    "source_package": "public_benchmark_reproduction_v1"
  },
  "benchmark": "MADRA_LIB_P1_P10_OPEN_RECONSTRUCTION",
  "benchmark_family": "madra",
  "benchmark_id": "MADRA_LIB_P1_P10_PAPER_SPEC_OPEN_RECONSTRUCTION_V1",
  "case_id": "MADRA_OPEN_P6_single",
  "claim_boundary": "Paper-spec open reconstruction; not the authors' unreleased P1-P10
      numerical instances.",
  "compiler_schema_version": "wta_mafa_agent_v0_4_clean.public_input_compiler.v1",
  "constraint_components": [
    {
      "evidence_ref": "public_case.objective",
      "formula_text": "For each target, selected radar-launcher engagements must not exceed
          max_attempts_per_targ ...",
      "id": "C001",
... (298 further lines omitted)
```


Appendix J.3. Planner output


```
{
  "artifact_type": "planner_output",
  "benchmark": "MADRA_LIB_P1_P10_OPEN_RECONSTRUCTION",
  "constraint_candidates": [
    {
      "formula_text": "For each target_id: sum(x[e]) <= max_attempts_per_target, where
          max_attempts_per_target = 1.",
      "id": "target_attempt_limit",
      "pattern_id": "assignment_attempt_limit",
      "scope": "target"
    },
    {
      "formula_text": "For each radar_id: sum(x[e]) <= public radar channel_capacity.",
      "id": "radar_channel_capacity",
      "pattern_id": "resource_capacity",
      "scope": "radar"
    },
    "... (2 more entries omitted)"
  ],
  "executable_risks": [
    {},
    {},
    "... (4 more entries omitted)"
  ],
  "forbidden_data_boundaries": [
    "{'boundary': 'hidden/reference/trusted-builder artifacts', 'path_pattern':
        'trusted_evaluation/**', 'reason': 'Restricted source root; no hidden reference or
        trusted builder artifacts may be read.'}",
    "{'boundary': 'hidden/reference answer key', 'path_pattern': '**/answer_key*', 'reason':
        'Non-public answer-key fields are forbidden.'}",
    "... (3 more entries omitted)"
  ],
  "model_scope_assessment": {
    "assessment_digest": "42079af31443ad392f1d2fe3978ba812b137ab3c9841e8bd130bffab5e35461d",
    "builder_directives": [
      "Implement only the Planner-approved modeling level: time_indexed_public.",
      "Do not introduce variables or constraints for unsupported constructs listed in
          forbidden_modeling_constructs.",
      "... (5 more entries omitted)"
    ],
```

```
    "forbidden_modeling_constructs": [
      {
        "blocked_terms": [
          "launch_time",
          "launch_step_decision"
        ],
        "name": "independent_launch_time_variable",
        "reason": "No public launch_step or launch_time_step field is materialized.",
        "route_hint": "modeler"
      },
      {
        "blocked_terms": [
          "trajectory",
          "kinematic",
          "... (4 more entries omitted)"
        ],
        "name": "continuous_trajectory",
        "reason": "No public trajectory, position, coordinate, or velocity field is materialized
            as decision inputs.",
        "route_hint": "modeler"
... (115 further lines omitted)
```


Appendix J.4. Model Design Card


```
{
  "artifact_type": "model_design_card",
  "assumptions": [
    {},
    {},
    "... (5 more entries omitted)"
  ],
  "benchmark": "MADRA_LIB_P1_P10_OPEN_RECONSTRUCTION",
  "benchmark_contract_bindings": {
    "binding_digest": "4b1b2bfe10e3d7d93f0cb0ff6b1160f6e01e739ced754c633f250914746fc7db",
    "candidate_sets": [
      {
        "authoritative": true,
        "binding_source": "computed_from_model_design_card",
        "bound": true,
        "filtering_allowed": false,
        "id": "E",
        "required_item_fields": [
          "target_id",
          "radar_id",
          "... (4 more entries omitted)"
        ]
      }
    ],
    "constraint_contracts": [
      {
        "binding_source": "computed_from_model_design_card",
        "bound_data_refs": [
          "public_instance_data.engagement_options"
        ],
        "constraint_id_bound": true,
        "id": "target_attempt_limit",
        "pattern_id": "assignment_attempt_limit",
        "required_data_refs": [
          "public_instance_data.engagement_options"
        ]
      },
      {
        "binding_source": "computed_from_model_design_card",
        "bound_data_refs": [
          "public_instance_data.engagement_options",
          "public_instance_data.radars"
        ],
        "constraint_id_bound": true,
        "id": "radar_channel_capacity",
        "pattern_id": "resource_capacity",
        "required_data_refs": [
          "public_instance_data.engagement_options",
          "public_instance_data.radars"
        ]
      },
      "... (2 more entries omitted)"
    ],
    "contract_digest": "15cd08c19dc020ae0b8d0dc66c2f5508f4ec5810e72ae9632551740f3f85d899",
    "contract_id": "bec.madra.MADRA_OPEN_P6_single.v1",
    "contract_projection": {
      "active_objectives": {
        "all_available_objective_ids": [
          "aggregate_interception_probability"
        ],
        "objective_count": 1,
        "objective_definitions": {
          "aggregate_interception_probability": {
            "description": "Sum of target-level cumulative interception probabilities under the
                public MADRA case definition.",
            "evidence_ref": "public_case.objective",
            "formula_text": "sum_i (1 - product_k(1 - p_ik))",
            "id": "aggregate_interception_probability",
```

```
            "name": "aggregate_interception_probability",
            "sense": "maximize",
            "uses": [
              "engagement_options",
              "interception_probability",
              "... (1 more entries omitted)"
            ]
          }
        },
        "objective_ids": [
          "aggregate_interception_probability"
        ],
... (328 further lines omitted)
```


Appendix J.5. Critic report


```
{
  "artifact_type": "critic_report",
  "checked_dimensions": [
    "executability",
    "public_data_fidelity",
    "objective_presence",
    "constraint_coverage",
    "family_field_alignment",
    "scope_alignment",
    "non_public_boundary"
  ],
  "defects": [],
  "design_card_digest": "f46842bcee2bd003623e4288052770253b79a4f90bfb6f5e13388c54b5be594d",
  "notes": [
    "P3 critic is a pre-build semantic gate, not the external benchmark evaluator.",
    "planner_digest=08e150a021b92c24a4fe539a3bdcfe09afa2ad35e24f749e61b9a65fadbf734f",
    "memory_card_digest="
  ],
  "passed": true,
  "public_input_digest": "781e84e41013d5d8c7db25cd48558db084b6f7de5ac44d45f347b47dbdfb16f5"
}
```


Appendix J.6. Error Router decision and route history


```
{
  "artifact_type": "error_route_decision",
  "decision_digest": "5d6a1965d5e7be8968f48246af1fdd13c18b0cdea356756d1d3e4a7b661d073a",
  "defects": [
    {
      "component_id": "",
      "details": {},
      "error_code": "BUILDER_SLOT_MISSING",
      "message": "LLM Builder slot response must include object field variable_slot_llm.",
      "owner": "variable_agent",
      "phase": "stage6_7_builder_slot_contract",
      "severity": "critical"
    },
    {
      "component_id": "",
      "details": {},
      "error_code": "BUILDER_SLOT_MISSING",
      "message": "LLM Builder slot response must include object field objective_slot_llm.",
      "owner": "objective_agent",
      "phase": "stage6_7_builder_slot_contract",
      "severity": "critical"
    },
    {
      "component_id": "",
      "details": {},
      "error_code": "BUILDER_SLOT_MISSING",
      "message": "LLM Builder slot response must include object field constraint_slot_llm.",
      "owner": "constraint_agent",
      "phase": "stage6_7_builder_slot_contract",
      "severity": "critical"
    },
    {
      "component_id": "",
      "details": {},
      "error_code": "BUILDER_SLOT_MISSING",
      "message": "LLM Builder slot response must include object field solution_view_slot_llm.",
      "owner": "solution_view_slot",
      "phase": "stage6_7_builder_slot_contract",
      "severity": "critical"
    },
... (18 further lines omitted)
```



```
[
  {
    "artifact_type": "error_route_decision",
    "decision_digest": "5d6a1965d5e7be8968f48246af1fdd13c18b0cdea356756d1d3e4a7b661d073a",
    "defects": [
      {
        "component_id": "",
        "details": {},
```

```
      "error_code": "BUILDER_SLOT_MISSING",
      "message": "LLM Builder slot response must include object field variable_slot_llm.",
      "owner": "variable_agent",
      "phase": "stage6_7_builder_slot_contract",
      "severity": "critical"
    },
    {
      "component_id": "",
      "details": {},
      "error_code": "BUILDER_SLOT_MISSING",
      "message": "LLM Builder slot response must include object field objective_slot_llm.",
      "owner": "objective_agent",
      "phase": "stage6_7_builder_slot_contract",
      "severity": "critical"
    },
    {
      "component_id": "",
      "details": {},
      "error_code": "BUILDER_SLOT_MISSING",
      "message": "LLM Builder slot response must include object field constraint_slot_llm.",
      "owner": "constraint_agent",
      "phase": "stage6_7_builder_slot_contract",
      "severity": "critical"
    },
    {
      "component_id": "",
      "details": {},
      "error_code": "BUILDER_SLOT_MISSING",
      "message": "LLM Builder slot response must include object field
          solution_view_slot_llm.",
      "owner": "solution_view_slot",
      "phase": "stage6_7_builder_slot_contract",
      "severity": "critical"
    },
    {
      "component_id": "",
      "details": {},
      "error_code": "BUILDER_SLOT_MISSING",
... (15 further lines omitted)
```


Appendix J.7. Failure report

The failure report is the artifact that keeps the run auditable: it names the defect, the responsible component, the phase, and what was retained.


```
{
  "accepted_source": "",
  "best_attempt_index": null,
  "best_available_artifact_dir": "",
  "claim_boundary": "External failure report preserves low-quality or repair-failed solver-
      success artifacts in the denominator; it must not be fed back to the same sample's
      internal Agent loop.",
  "defects": [
    {
      "component_id": "",
      "details": {},
      "error_code": "BUILDER_SLOT_MISSING",
      "message": "LLM Builder slot response must include object field variable_slot_llm.",
      "owner": "variable_agent",
      "phase": "stage6_7_builder_slot_contract",
      "severity": "critical"
    },
    {
      "component_id": "",
      "details": {},
      "error_code": "BUILDER_SLOT_MISSING",
      "message": "LLM Builder slot response must include object field objective_slot_llm.",
      "owner": "objective_agent",
      "phase": "stage6_7_builder_slot_contract",
      "severity": "critical"
    },
    {
      "component_id": "",
      "details": {},
      "error_code": "BUILDER_SLOT_MISSING",
      "message": "LLM Builder slot response must include object field constraint_slot_llm.",
      "owner": "constraint_agent",
      "phase": "stage6_7_builder_slot_contract",
      "severity": "critical"
    },
    {
      "component_id": "",
      "details": {},
      "error_code": "BUILDER_SLOT_MISSING",
      "message": "LLM Builder slot response must include object field solution_view_slot_llm.",
      "owner": "solution_view_slot",
      "phase": "stage6_7_builder_slot_contract",
      "severity": "critical"
    },
    "... (1 more entries omitted)"
  ],
```

```
  "failure_report_digest": "00302608ed1becbfd6e041a8d25c70a3fa11ad61295926992db25a472d213abf",
  "failure_type": "BUILD_FAILURE",
... (9 further lines omitted)
```


Appendix J.8. Retained mathematical description

Although no executable model was produced, the formulation reached before the failure is exported and retained. Note what the export records: the trust status is `unverified_or_failed_run`, and the builder-code and model-registry digests are `unavailable`. The constraint block is declared rather than instantiated, because construction never completed. A failed run therefore yields an artifact that states plainly what it is, instead of a formulation that looks finished.

Instance: MADRA / MADRA_OPEN_P6_single
Objective contract: `madra.value_weighted_interception`
Math trust status: `unverified_or_failed_run`
Source mode: `benchmark_execution_contract_madra_unverified`

Executable evidence.

Workspace digest	`421e2fc717ab60b7`
Slot digest	`ca9aea40af72036a`
Builder-code digest	`unavailable`
Model-registry digest	`unavailable`
BEC digest	`15cd08c19dc020ae`
External objective-report digest	`78e8f5eced5f9ab5`
LLM reinterpretation used	`false`
Feedback to the current agent allowed	`false`

MADRA contract-level formulation (unverified).

$$\max_{x} \quad \sum_{e \in E} V_{t(e)} P_e x_e$$

$$\text{s.t.} \quad \text{public radar, launcher, inventory and temporal rules}$$

$$\text{as declared by the execution contract}$$

$$x_e \in \{0,1\} \qquad \forall e \in E.$$

Symbols. E is the authoritative public engagement option set; $V_{t(e)}$ is defended value; P_e is the public interception probability or effectiveness.

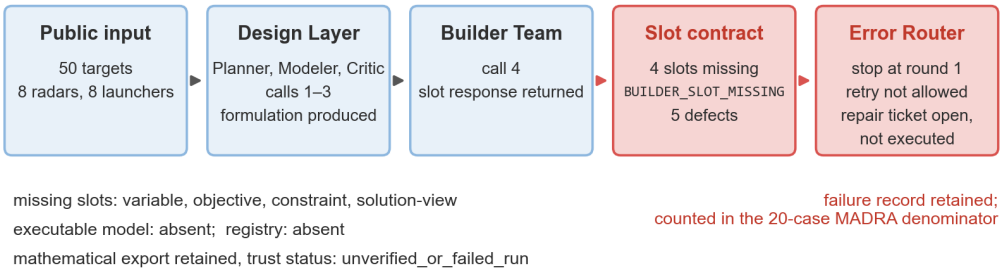


Figure J.1: Case II: MADRA trace stopped at the first route decision.

Appendix K. Released Material

WTA-FormBench and this appendix are released at `https://anonymous.4open.science/r/WTA-MAFA2026-C583/`. The release contains the frozen benchmark, namely the agent-visible input of all 144 samples, the contract schemas and the frozen validator, together with this appendix as a PDF.

The aggregates of the main paper are reported per stage in Appendix E, per quality dimension in Table E.3, per failure endpoint in Table E.5, and per family in Appendix H.6 and Appendix H.7. Failed samples remain in every denominator, and the two complete run traces of Appendix I and Appendix J are reproduced in this appendix rather than only indexed.

Evaluator-private benchmark material is deliberately withheld: the agents never receive the reference formulations, validation traces, assessability masks or the canonical element registry, and releasing them would let a later system be tuned against the answers and invalidate WTA-FormBench for its users.

References

[1] Zheng N, Wang R, Li K, Li W, Luo T, Wang H. Large-scale multi-objective sensor-weapon-target assignment optimization: Benchmark suites and knowledge-driven heuristics. IEEE Trans Evol Comput. 2026:1-1. `https://doi.org/10.1109/TEVC.2026.3683727`.

[2] Li W, Wang R, Heng Y, Zhang T, Wang L. Knowledge-guided evolutionary optimization for large-scale air defense resource allocation. IEEE Trans Artif Intell. 2024;5(12):6267-79. https://doi.org/10.1109/TAI.2024.3375263.
[3] Zhang B, Luo P, Yang G, Soong BH, Yuen C. OR-LLM-Agent: Automating modeling and solving of operations research optimization problems with reasoning LLM. arXiv [Preprint]. 2025. https://doi.org/10.48550/arXiv.2503.10009.
[4] Xiao Z, Zhang D, Wu Y, Xu L, Wang Y, Han X, et al. Chain-of-Experts: When LLMs meet complex operations research problems. In: International Conference on Learning Representations; 2024. p. 48519-37.
[5] AhmadiTeshnizi A, Gao W, Brunborg H, Talaei S, Lawless C, Udell M. OptiMUS-0.3: Using large language models to model and solve optimization problems at scale. arXiv [Preprint]. 2024. https://doi.org/10.48550/arXiv.2407.19633.
[6] Xie G. ORPilot: A production-oriented agentic LLM-for-or tool for optimization modeling. arXiv [Preprint]. 2026. https://doi.org/10.48550/arXiv.2605.02728.
[7] Liu F, Yang Z, Liu C, Song T, Gao X, Liu H. MM-Agent: LLM as agents for real-world mathematical modeling problem. In: Advances in Neural Information Processing Systems; 2025. p. 23677-730. https://doi.org/10.52202/085713-0704.
[8] Qian C, Du H, Wang H, Chen X, Zhang Y, Sil A, et al. ModelingAgent: Bridging LLMs and mathematical modeling for real-world challenges. In: Findings of the Association for Computational Linguistics: EMNLP; 2025. p. 1599-633. https://doi.org/10.18653/v1/2025.findings-emnlp.85.
[9] Wang Y, Wang J, Hao JK, Feng J. Efficient adaptive large neighborhood search for sensor-weapon-target assignment. IEEE Trans Syst Man Cybern Syst. 2024;54(10):6397-409. https://doi.org/10.1109/TSMC.2024.3431033.
[10] Yi X, Yu H, Xu T. Solving multi-objective weapon-target assignment considering reliability by improved MOEA/D-AM2M. Neurocomputing. 2024;563:126906. https://doi.org/10.1016/j.neucom.2023.126906.
[11] Yang B, Wang T, Li B, Luo F, Gao C. A vehicle-target assignment framework: Collaborative missions of multiple high-speed flight vehicles with multiple targets. IEEE Trans Aerosp Electron Syst. 2025;61(6):16403-18. https://doi.org/10.1109/TAES.2025.3595106.
[12] Arslan C, Karasakal O, Kırca Ö. Solution approaches for the dynamic naval air defense planning problem. IEEE Trans Syst Man Cybern Syst. 2026;56(3):2189-201. https://doi.org/10.1109/TSMC.2025.3648686.
[13] Chang X, Shi J, Luo Z, Liu Y. Adaptive large neighborhood search algorithm for multi-stage weapon target assignment problem. Comput Ind Eng. 2023;181:109303. https://doi.org/10.1016/j.cie.2023.109303.
[14] Tashakori S, Ranjbar M, Balochian S, Sharif-Razavian J, Peymankar M. Dynamic soft-kill weapon-target assignment in naval environments. Comput Ind Eng. 2024;197:110606. https://doi.org/10.1016/j.cie.2024.110606.
[15] He G, Chen A, Xuan L, Li X, Chang H, Yuan G, et al. Modelling and simulation of distributed target allocation based on improved NSGA-II in multi-UUV collaborative attack. Appl Soft Comput. 2026;186:114174. https://doi.org/10.1016/j.asoc.2025.114174.
[16] Li W, Yao X, Li K, Wang R, Zhang T. Knowledge-guided competitive evolutionary algorithm for multi-solution sensor-weapon-target assignment problem. IEEE Trans Evol Comput. 2026. https://doi.org/10.1109/TEVC.2026.3653800.
[17] Ma Y, Wang G, Hu X, Luo H. Two-stage hybrid heuristic search algorithm for novel weapon target assignment problems. Comput Ind Eng. 2021;162:107717. https://doi.org/10.1016/j.cie.2021.107717.
[18] Luo W, Lü J, Liu K, Chen L. Learning-based policy optimization for adversarial missile-target assignment. IEEE Trans Syst Man Cybern Syst. 2022;52(7):4426-37. https://doi.org/10.1109/TSMC.2021.3096997.
[19] Zheng N, Wang R, Li K, Li W, Luo T, Deng H. Balancing global and local search via Q-Learning in evolutionary algorithms for air defense resource assignment problems. IEEE Trans Syst Man Cybern Syst. 2026;56(8):4891-904. https://doi.org/10.1109/TSMC.2026.3690023.
[20] Han R, Zhang T, Zhong H, Wang Y. A dynamic UAVs cooperative suppressive jamming method with joint task assignment and bandwidth allocation. IEEE Trans Aerosp Electron Syst. 2025;61(5):11469-88. https://doi.org/10.1109/TAES.2025.3566932.
[21] Bai Z, Zhou H, Shi J, Xing L, Wang J. A hybrid multi-objective evolutionary algorithm with high solving efficiency for UAV defense programming. Swarm Evol Comput. 2024;87:101572. https://doi.org/10.1016/j.swevo.2024.101572.
[22] Li N, Su Z, Ling H, Karatas M, Zheng Y. Optimization of air defense system deployment against reconnaissance drone swarms. Complex Syst Model Simul. 2023;3(2):102-17. https://doi.org/10.23919/CSMS.2023.0003.
[23] Hendrickson K, Ganesh P, Volle K, Buzaud P, Brink K, Hale M. Decentralized weapon-target assignment under asynchronous communications. J Guid Control Dyn. 2023;46(2):312-24. https://doi.org/10.2514/1.G006532.
[24] Liu C, Li J, Wang Y, Yu Y, Guo L, Gao Y, et al. A time-driven dynamic weapon target assignment method. IEEE Access. 2023;11:129623-39. https://doi.org/10.1109/ACCESS.2023.3332513.
[25] Kim JE, Lee CH, Yi MY. New weapon target assignment algorithms for multiple targets using a rotational strategy and clustering approach. IEEE Access. 2022;10:43738-50. https://doi.org/10.1109/ACCESS.2022.3168718.